

MODUL PRAKTIKUM
EL2008 PEMECAHAN MASALAH DENGAN PEMROGRAMAN



Oleh

Dr. Reza Darmakusuma, S.T, M.T.

Dr. Rahadian Yusuf, S.T., M.T.

Ishak Hilton Pujantoro Tnunay, S.T., Ph.D.

Dimas Widyanto

Elkhan Julian Brillianshah

Irfan Tito Kurniawan

Stefen Sutandi

PROGRAM STUDI TEKNIK ELEKTRO
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

FLOWCHART	6
I. Kaidah Flowchart	7
I.I. Blok Start/End	7
I.II. Blok Input/Output	7
I.III. Blok Decision	7
I.IV. Blok Process	8
I.V. Blok Predefined Process	8
I.VI. Blok Reference	8
II. Contoh Flowchart	9
DATAFLOW DIAGRAM	11
I. Kaidah DFD	11
I.I. External Entity	12
I.II. Process	12
I.III. Data Store	12
I.IV. Data Flow	12
I.V. Pembuatan DFD	13
II. Contoh DFD	13
ANALISIS KOMPLEKSITAS RUANG DAN WAKTU	14
I. Analisis Kompleksitas Ruang	14
II. Analisis Kompleksitas Waktu	14
III. Contoh Analisis Kompleksitas Ruang dan Waktu	15
MODUL I : OVERVIEW OF C LANGUAGE	18
I. Tujuan	18
II. Materi	18
II.I. Basic Structures	18
II.II. Variable Declaration and Data Type	20
II.III. Operator	21
II.IV. Branch	25
II.V. Looping	29
II.VI. Function	30
II.VII. Recursions	32
III. Tutorial	33
MODUL II : DATA MANIPULATION AND EXTERNAL FILES	35
I. Tujuan	35

II. Materi	35
II.I. Array	35
II.II. String	36
II.III. Pointer	38
II.IV. File Eksternal	39
II.V. Passing Parameter	41
III. Tutorial	43
MODUL III : STRUCTURES AND DYNAMIC ARRAYS	47
I. Tujuan.....	47
II. Materi	47
II.I. Array Dinamis	47
II.II. Structure	48
III. Tutorial	50
MODUL IV : DYNAMIC STRUCTURES	52
I. Tujuan.....	52
II. Materi	52
II.I. Linked List	52
II.II. Stack.....	54
II.III. Queue	55
III. Tutorial	56
MODUL V : FOUNDATION OF ALGORITHM.....	60
I. Tujuan.....	60
II. Materi	60
II.I. Graph and Tree.....	60
II.II. Strategi Algoritma	62
III. Tutorial	63

DAFTAR TABEL

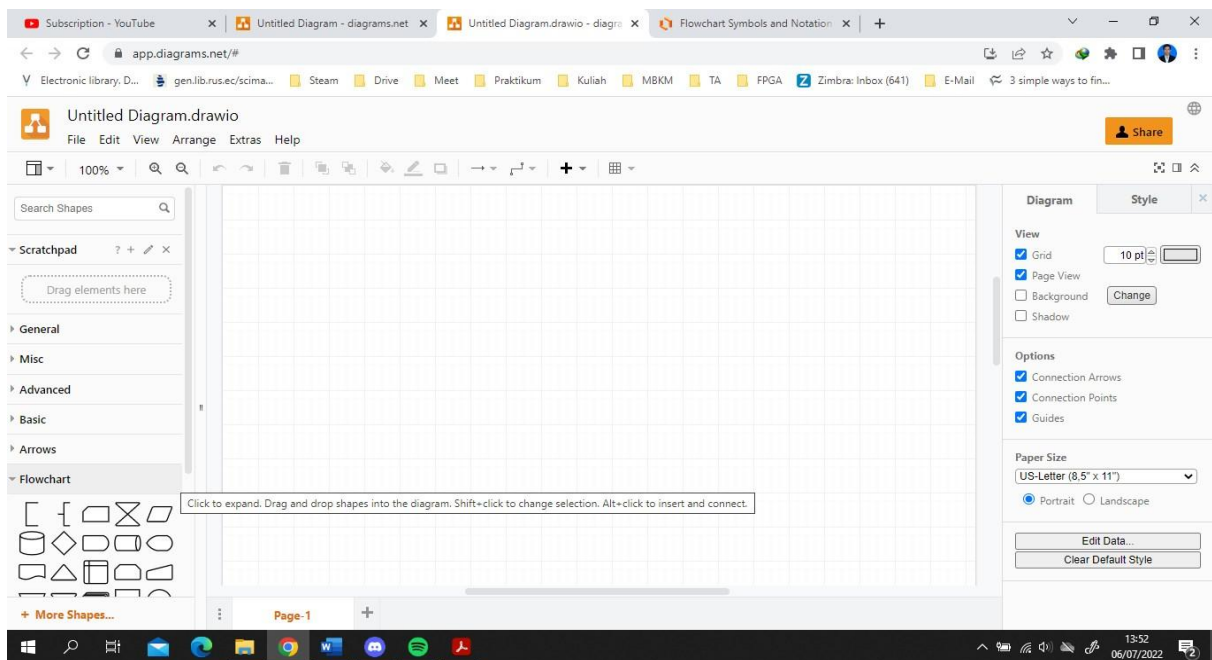
Tabel 1 Tipe Data Integer.....	20
Tabel 2 Tipe Data Floating-Point.....	20
Tabel 3 Arithmetic Operators.....	22
Tabel 4 Relational Operators.....	22
Tabel 5 Logical Operators.....	23
Tabel 6 Bitwise Operators.....	23
Tabel 7 Assignment Operators.....	24
Tabel 8 Operator Penting Lainnya	25
Tabel 9 Contoh Fungsi dalam Library String.h.....	37
Tabel 10 Mode Pembacaan File Eksternal dalam Bahasa C.....	40
Tabel 11 File Eksternal Tutorial Modul 2	46
Tabel 12 Fungsi Pengolahan Array Dinamis dalam Bahasa C.....	47

DAFTAR GAMBAR

Gambar 1 Tampilan pada Draw.io	6
Gambar 2 Blok Start (kiri) dan Blok End (kanan)	7
Gambar 3 Blok Input/Output	7
Gambar 4 Blok Decision.....	8
Gambar 5 Blok Process.....	8
Gambar 6 Blok Predefined Process	8
Gambar 7 Blok Reference.....	8
Gambar 8 Contoh Flowchart dari Kode	10
Gambar 9 Berbagai Notasi Simbol DFD	11
Gambar 10 Contoh DFD Level 0 atau Context Diagram.....	13
Gambar 11 Contoh DFD Level 1	13
Gambar 12 Proses Kompilasi pada Bahasa C.....	18
Gambar 13 Struktur Kondisional	25
Gambar 14 Struktur Switch Statement.....	27
Gambar 15 Struktur Perulangan.....	29
Gambar 16 Ilustrasi Array.....	35
Gambar 17 Ilustrasi Array dengan Alamat Memori.....	52
Gambar 18 Ilustrasi Linked List dengan Alamat Memori	52
Gambar 19 Ilustrasi Stack	54
Gambar 20 Ilustrasi Queue	55
Gambar 21 Ilustrasi Graph.....	60
Gambar 22 Ilustrasi Tree.....	61

FLOWCHART

Secara umum pembuatan flowchart dibebaskan dengan catatan mengikuti kaidah pada umumnya, namun sangat disarankan untuk pembuatan flowchart menggunakan aplikasi khusus untuk membuat flowchart seperti draw.io (<https://app.diagrams.net/>) atau lucidchart (<https://www.lucidchart.com/>). Dalam dokumen ini akan digunakan aplikasi draw.io dalam pembuatan flowchart. Silakan akses aplikasi melalui tautan yang disediakan kemudian gunakan menu flowchart yang tersedia pada bagian kiri tampilan seperti pada Gambar 1.



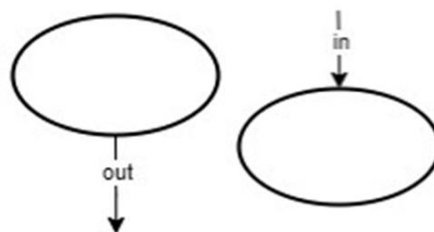
Gambar 1 Tampilan pada Draw.io

I. Kaidah Flowchart

Terdapat enam buah bentuk blok yang perlu Anda perhatikan dalam membuat *flowchart*. Masing-masing blok memiliki kegunaan dan kaidah yang berbeda-beda. Berikut merupakan penjelasan dari masing-masing blok.

I.I. Blok Start/End

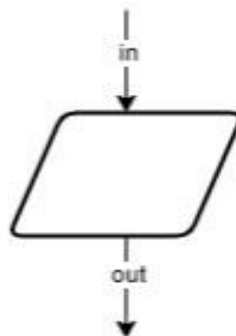
Blok ini digunakan untuk mengawali atau mengakhiri sebuah flowchart. Blok ini memiliki masing-masing satu input atau satu output saja. Bentuk yang digunakan adalah oval dengan satu output untuk blok end (kanan) dan satu input untuk blok start (kiri).



Gambar 2 Blok Start (kiri) dan Blok End (kanan)

I.II. Blok Input/Output

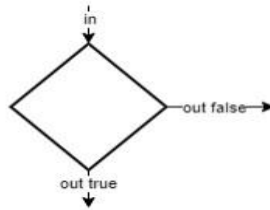
Blok input/output digunakan untuk mendefinisikan proses input/output yang dilakukan pada program. Blok ini berbentuk jajar genjang dengan satu buah input dan satu buah output.



Gambar 3 Blok Input/Output

I.III. Blok Decision

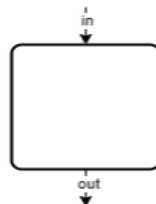
Blok decision merupakan blok yang digunakan untuk menggambarkan proses kondisional yang terdapat pada program. Blok ini juga digunakan untuk menggambarkan proses perulangan. Blok decision berbentuk belah ketupat dengan satu buah input dan dua buah output. Letak output true dan output false tidak harus sama dengan contoh yang diberikan tetapi dapat disesuaikan dengan kebutuhan, masih jelas terdapat satu buah input dan dua buah output pada blok ini.



Gambar 4 Blok Decision

I.IV. Blok Process

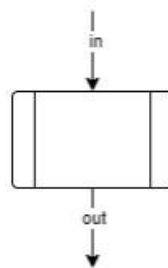
Blok process merupakan blok yang digunakan untuk merepresentasikan sebuah proses yang dilakukan oleh program. Blok ini berbentuk persegi dengan sebuah input dan sebuah output.



Gambar 5 Blok Process

I.V. Blok Predefined Process

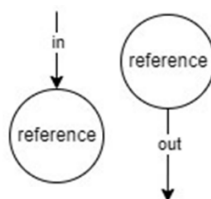
Blok ini merupakan blok yang digunakan untuk melambangkan sebuah fungsi buatan yang digunakan pada program utama. Blok ini memiliki bentuk persegi dengan tambahan dua garis di bagian dalam serta sebuah input dan sebuah output. Fungsi yang digunakan pada blok ini juga perlu digambarkan flowchart-nya pada bagian terpisah dari fungsi utama.



Gambar 6 Blok Predefined Process

I.VI. Blok Reference

Blok ini digunakan sebagai pemisah flowchart apabila terlalu panjang. Blok ini berbentuk lingkaran berisi nomor atau huruf referensi, terdiri dari sebuah input atau sebuah output.



Gambar 7 Blok Reference

II. Contoh Flowchart

Diberikan contoh kode yang ditampilkan pada Source Code 1 berikut. Dengan kode tersebut akan dihasilkan *flowchart* yang sesuai pada Gambar 8. Kode yang diberikan merupakan kode yang memiliki sebuah fungsi untuk melakukan penjumlahan. Kode tersebut akan digunakan untuk melakukan penjumlahan dari setiap elemen ganjil dalam sebuah *array*.

Source Code 1 Contoh Kode untuk Pembuatan Flowchart

```
/* EL2008 Praktikum Pemecahan Masalah dengan Pemrograman
 * Modul      : Contoh kode untuk Flowchart
 * Pembuat    : Stefen Sutandi
 * Deskripsi   : Program untuk melakukan penjumlahan pada setiap bilangan ganjil dalam
sebuah array
 */

#include <stdio.h>

// Fungsi untuk menjumlahkan dua bilangan
int sum(int a, int b) {
    int hasil = a + b;
    return hasil;
}

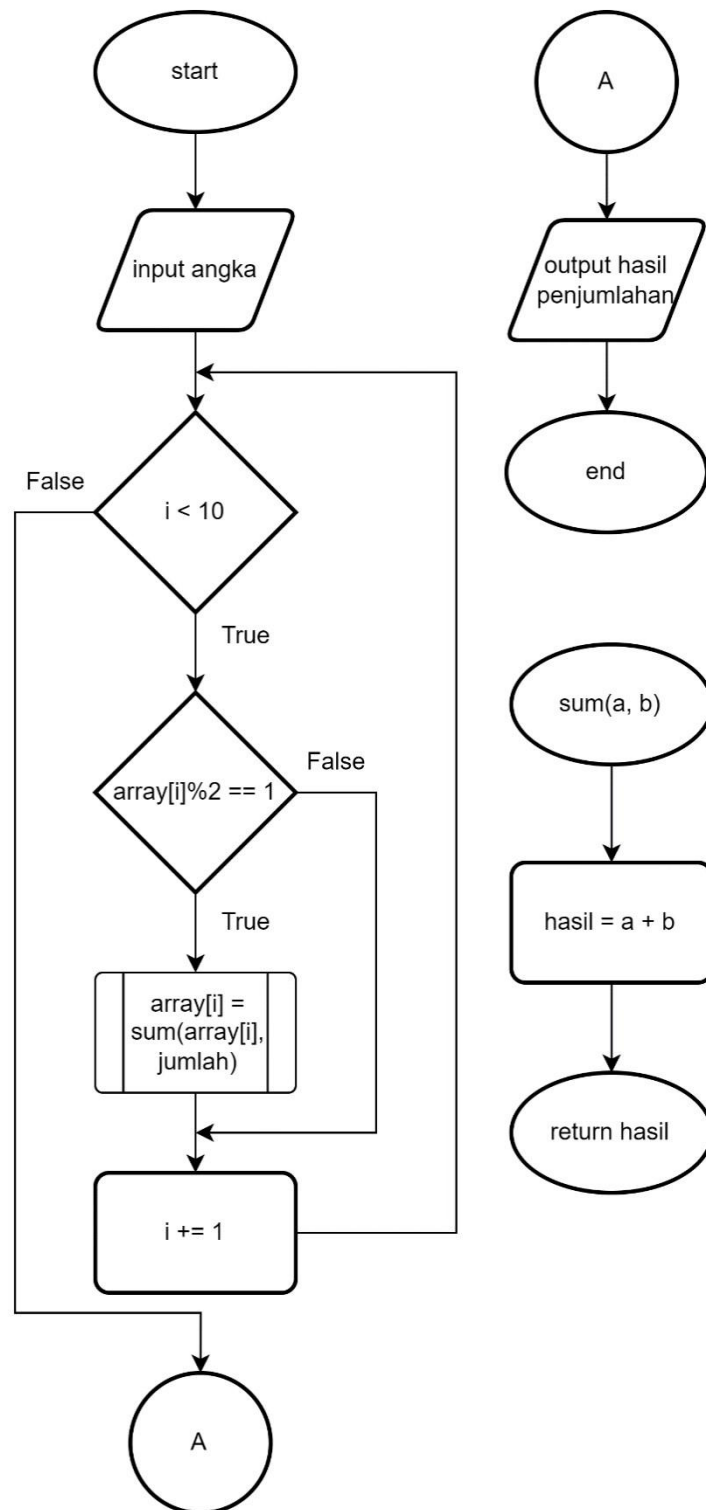
int main() {
    int array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int jumlah;

    // Input nilai jumlah
    printf("Masukkan angka yang akan dijumlahkan: ");
    scanf("%d", &jumlah);

    // Menambahkan nilai ke bilangan ganjil dalam array
    for (int i = 0; i < 10; i++) {
        if (array[i] % 2 == 1) {
            array[i] = sum(array[i], jumlah);
        }
    }

    // Output hasil array
    printf("Hasil:\n");
    for (int i = 0; i < 10; i++) {
        printf("%d\n", array[i]);
    }
    return 0;
}
```

Flowchart yang dibuat terdiri dari dua buah bagian yaitu bagian `main` dan bagian fungsi `sum`. Perlu diperhatikan bahwa pemanggilan fungsi `sum` pada bagian `main` akan menggunakan blok predefined process. Flowchart pada bagian `main` juga diberikan contoh penggunaan blok reference untuk memberi tanda apabila sebuah flowchart terlalu panjang.



Gambar 8 Contoh Flowchart dari Kode

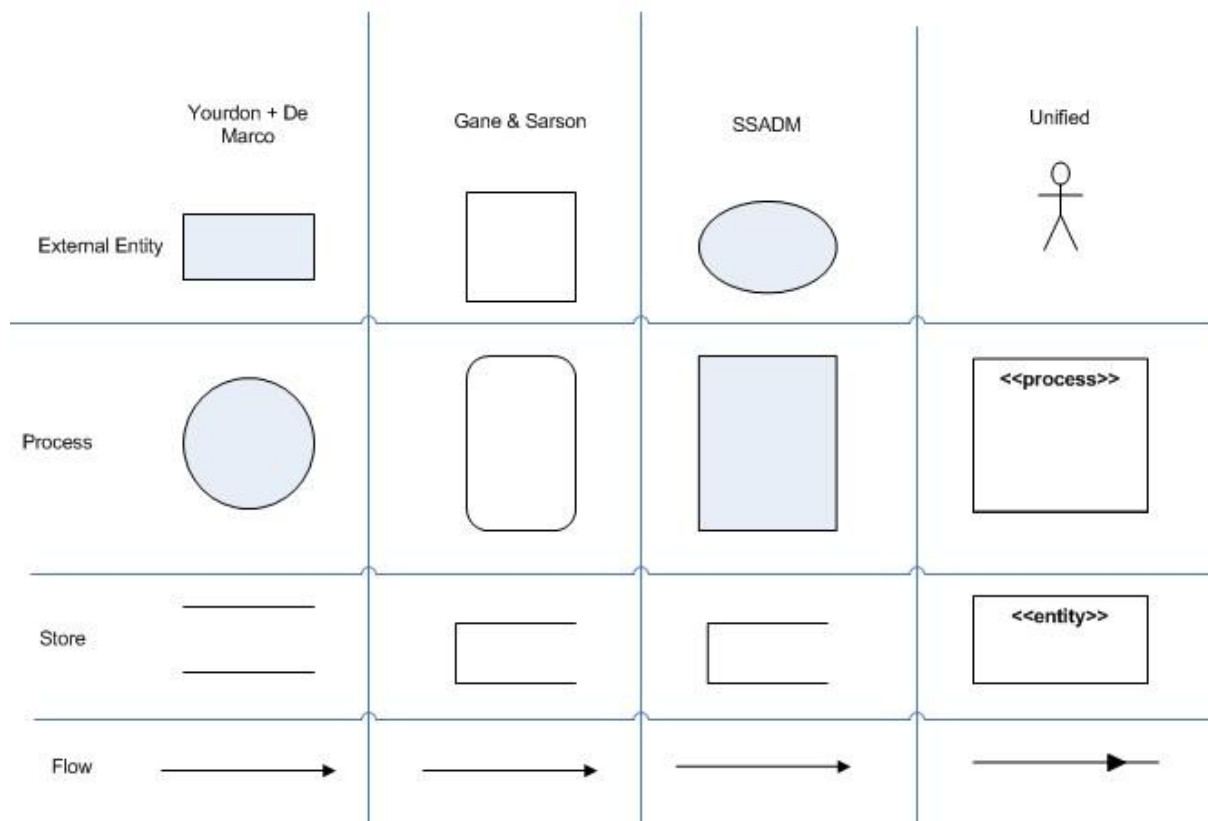
Perhatikan bahwa setiap blok pada flowchart hanya memiliki satu input dan output kecuali blok kondisional yang memiliki dua output. Perhatikan pula bahwa blok Start/End pada flowchart fungsi sum memuat parameter fungsi serta nilai yang di-return oleh fungsi tersebut.

DATAFLOW DIAGRAM

Sama seperti flowchart, pembuatan Data Flow Diagram atau DFD dibebaskan dengan catatan mengikuti kaidah pada umumnya, namun sangat disarankan untuk pembuatan DFD menggunakan aplikasi seperti draw.io (<https://app.diagrams.net/>) atau lucidchart (<https://www.lucidchart.com/>). Dalam dokumen ini akan digunakan aplikasi draw.io dalam pembuatan DFD.

I. Kaidah DFD

Terdapat empat buah simbol yang perlu Anda perhatikan dalam membuat DFD. Setiap simbol merepresentasikan hal yang berbeda-beda dan juga memiliki kegunaan dan kaidah yang berbeda-beda. Simbol-simbol tersebut dapat digambarkan dengan beberapa cara seperti pada Gambar 8.



Gambar 9 Berbagai Notasi Simbol DFD

Notasi DFD yang dapat Anda gunakan pada praktikum ini adalah Yourdon & De Marco atau Gane & Sarson. Berikut merupakan penjelasan dari masing-masing simbol.

I.I. External Entity

External Entity merupakan sebuah simbol yang menggambarkan orang, departemen, atau sistem eksternal yang memberikan data kepada program atau menerima data dari program. Simbol ini akan menggambarkan cara program Anda berinteraksi dengan dunia luar. Hal yang digambarkan oleh simbol ini merupakan hal berada di luar program Anda seperti pengguna program.

I.II. Process

Sesuai dengan namanya, simbol Process menggambarkan sebuah proses dalam program Anda yang menerima masukkan dan memberikan keluaran dengan bentuk atau konten yang berbeda. Simbol ini dapat menggambarkan satu proses sederhana atau beberapa proses sekaligus. Kompleksitas proses yang digambarkan oleh simbol ini akan dijelaskan pada bagian selanjutnya.

I.III. Data Store

Simbol ini menggambarkan sebuah tempat penyimpanan data yang akan digunakan oleh satu atau beberapa proses di masa depan. Dalam konteks praktikum ini, sebuah file eksternal yang digunakan oleh program dapat digambarkan sebagai sebuah Data Store dalam DFD Anda.

I.IV. Data Flow

Data Flow merupakan simbol yang merepresentasikan jalur pergerakan data dalam program Anda. Simbol ini digambarkan dengan sebuah panah dengan teks yang menyatakan data yang melalui jalur tersebut. Terdapat beberapa peraturan dalam menggambarkan Data Flow, peraturan-peraturan tersebut adalah:

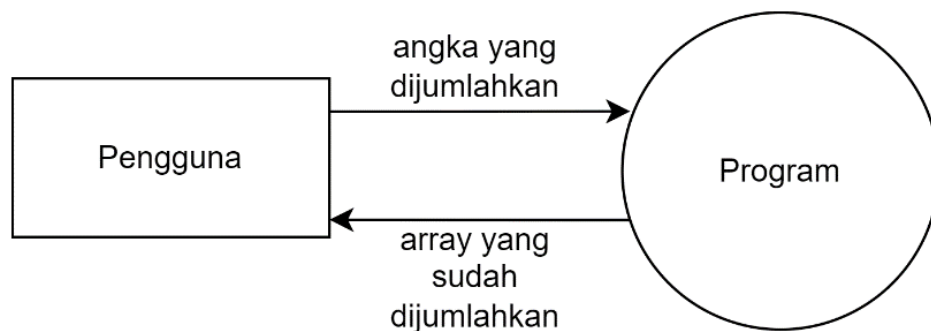
1. External Entity tidak dapat memberikan data ke External Entity lain tanpa melalui sebuah Process
2. External Entity tidak dapat memberikan data ke sebuah Data Store tanpa melalui sebuah Process
3. Data Store tidak dapat memberikan data kepada sebuah External Entity tanpa melalui sebuah Process
4. Data Store tidak dapat mengirim atau menerima data dari Data Store lain tanpa melalui sebuah Process

IV. Pembuatan DFD

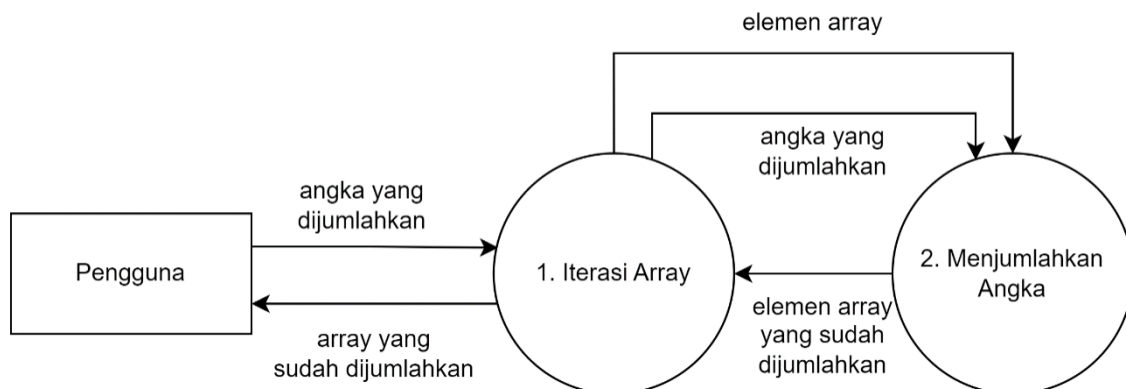
DFD dibuat dalam beberapa level yang dimulai dari level 0. DFD level 0 disebut juga dengan Context Diagram. DFD level 0 menggambarkan interaksi program dengan dunia luar sehingga hanya berisikan External Entity dan satu Process yang merangkum semua proses program. Process pada DFD level 0 akan dipecah menjadi beberapa Process lainnya di DFD level 1. Perlu diingat bahwa Data Flow pada DFD level 1 harus konsisten dengan Data Flow pada DFD level 0. Jika diperlukan, Process-Process pada DFD level 1 dapat dipecah lebih lanjut menjadi beberapa Process lainnya di DFD level 2.

II. Contoh DFD

Dengan kode Source Code 1, dapat dihasilkan DFD level 0 dan DFD level 1 menggunakan notasi Yourdan & DeMarco seperti pada Gambar 10 dan Gambar 11. Perhatikan bahwa DFD menggambarkan aliran data pada program, proses-proses yang ada dalam program, serta interaksi program dengan dunia luar



Gambar 10 Contoh DFD Level 0 atau Context Diagram



Gambar 11 Contoh DFD Level 1

ANALISIS KOMPLEKSITAS RUANG DAN WAKTU

Analisis kompleksitas ruang dan waktu merupakan bagian dari analisis algoritma yang dilakukan secara teoritis (apriori). Analisis algoritma dilakukan untuk menentukan kompleksitas dari sebuah algoritma berdasarkan jumlah waktu, storage, atau resource lainnya yang dibutuhkan untuk menjalankan algoritma tersebut hingga selesai. Sesuai dengan namanya, kompleksitas ruang merepresentasikan ukuran ruang memory yang digunakan oleh sebuah algoritma selama siklus hidupnya sedangkan kompleksitas waktu merepresentasikan jumlah waktu yang dibutuhkan untuk menjalankan sebuah algoritma sampai selesai. Kompleksitas ruang dan waktu dapat dinotasikan menggunakan beberapa cara, namun notasi yang digunakan pada praktikum ini adalah notasi Big O.

I. Analisis Kompleksitas Ruang

Analisis kompleksitas ruang dapat dilakukan dengan langkah-langkah berikut:

1. Menghitung ruang memory yang dialokasikan secara statis
2. Menghitung ruang memory yang dialokasikan secara dinamis (Auxiliary Space)
3. Menghitung total ruang yang dibutuhkan
4. Menyatakan analisis menggunakan notasi Big O

Ruang yang dialokasikan secara statis berasal dari deklarasi variabel-variabel yang bersifat pasti (contoh: variabel yang tidak dideklarasikan di dalam blok kondisional atau loop). Ruang yang dialokasikan secara dinamis berasal dari deklarasi variabel yang tidak pasti atau ukurannya dapat berubah selama eksekusi algoritma (contoh: alokasi memory di stack ketika memanggil fungsi, deklarasi variabel dalam blok kondisional, alokasi memory dinamis, dll.).

II. Analisis Kompleksitas Waktu

Analisis kompleksitas waktu dapat dilakukan dengan langkah-langkah berikut:

1. Menghitung berapa kali statement-statement dalam program dieksekusi
2. Menyatakan analisis menggunakan notasi Big O

Mirip dengan analisis kompleksitas ruang, jumlah eksekusi dapat bernilai statis maupun dinamis. Jumlah eksekusi statement dapat bernilai dinamis karena berada di dalam sebuah loop atau karena berada di dalam sebuah fungsi rekursif.

III. Contoh Analisis Kompleksitas Ruang dan Waktu

Sebagai contoh, akan dilakukan analisis kompleksitas ruang dan waktu untuk kode Source Code 1 yang merupakan modifikasi dari Source Code 1. Dengan itu, analisis dapat dilakukan seperti pada subbab-subbab berikut.

Source Code 2 Contoh Kode untuk Analisis Kompleksitas Ruang dan Waktu

```
/* EL2008 Praktikum Pemecahan Masalah dengan Pemrograman
 * Modul      : Contoh kode untuk Flowchart
 * Pembuat    : Elkhan Julian Brillianshah
 * Deskripsi   : Program untuk melakukan penjumlahan pada setiap
 *              bilangan ganjil dalam sebuah array
 */

#include <stdio.h>

// Fungsi untuk menjumlahkan dua bilangan
int sum(int a, int b) {
    int hasil = a + b;
    return hasil;
}

int main() {
    int N, jumlah;

    // Input jumlah elemen dalam array
    printf("Masukkan jumlah angka yang akan dimasukkan: ");
    scanf("%d", &N);

    int array[N];

    // Mengisi array dengan angka 1 hingga N
    for (int i = 0; i < N; i++) {
        array[i] = i + 1;
    }

    // Input angka yang akan ditambahkan ke bilangan ganjil
    printf("Masukkan angka yang akan dijumlahkan: ");
    scanf("%d", &jumlah);

    // Menambahkan angka ke elemen ganjil dalam array
    for (int i = 0; i < N; i++) {
        if (array[i] % 2 == 1) {
            array[i] = sum(array[i], jumlah);
        }
    }

    // Output hasil array
    printf("Hasil:\n");
    for (int i = 0; i < N; i++) {
        printf("%d\n", array[i]);
    }

    return 0;
}
```

1. Analisis Kompleksitas Ruang

Analisis kompleksitas ruang dimulai dengan menghitung ruang memory yang digunakan program dan dialokasikan secara statis. Berdasarkan kode, dapat diamati bahwa variabel yang pasti akan dideklarasikan adalah variabel jumlah, i , dan N saja. Untuk menyederhanakan perhitungan, akan diasumsikan bahwa tipe data `int` memiliki ukuran 1 satuan ruang sehingga total ruang yang dialokasikan secara statis adalah 3 satuan ruang.

Setelah itu, akan dilakukan perhitungan ruang memory yang dialokasikan secara dinamis. Berdasarkan kode, dapat diamati bahwa satu-satunya variabel yang ukurannya tidak pasti adalah variabel array saja yang memiliki panjang sebesar N sehingga berukuran $N * 1$. Selain itu, terdapat variabel hasil yang dideklarasikan di dalam fungsi `sum`. Dengan itu, didapat total ruang yang dialokasikan secara dinamis adalah $N + 1$ satuan ruang.

Total ruang yang dibutuhkan dapat dihitung dengan menjumlahkan ruang yang dialokasikan secara statis dan dinamis. Berdasarkan perhitungan-perhitungan sebelumnya, didapat bahwa total kebutuhan ruang adalah $N + 4$ satuan ruang. Notasi Big O dari hasil tersebut dapat dibuat dengan mengambil suku dengan derajat terbesar dan menghilangkan koefisien dari suku tersebut sehingga didapat kompleksitas ruang dalam notasi Big O adalah $O(N)$.

2. Analisis Kompleksitas Waktu

Analisis kompleksitas waktu dilakukan dengan menghitung berapa kali statement- statement dalam program dieksekusi. Demi kesederhanaan, fungsi `scanf` dan `printf` diasumsikan akan memiliki waktu eksekusi yang konstan sehingga dapat diperlakukan sebagai satu buah statement. Jika loop-loop dalam fungsi `main` dihiraukan, maka didapat bahwa terdapat 7 baris statement yang dieksekusi hanya sekali saja (deklarasi variabel, fungsi `printf`, dan fungsi `scanf`).

Pada for loop pertama dalam fungsi `main`, dapat diamati bahwa assignment elemen dari variabel array akan dilakukan sebanyak N kali sehingga bagian tersebut dapat dianggap akan mengeksekusi statement sebanyak N kali. For loop kedua di dalam fungsi `main` berisikan dua buah statement (statement `if` dan statement assignment), namun terdapat eksekusi fungsi `sum` yang akan mengeksekusi 3 buah statement sehingga bagian tersebut dapat dianggap akan mengeksekusi statement sebanyak $N * (1 + 1 + 3)$ atau $5N$ kali. Isi dari loop terakhir pada fungsi `main` merupakan eksekusi dari fungsi `printf` saja sehingga dapat dianggap bahwa bagian

tersebut akan mengeksekusi statement tersebut sebanyak N kali. Dengan itu, didapat bahwa program secara keseluruhan akan mengeksekusi statement $7N + 7$ kali yang dapat ditulis dalam notasi Big O sebagai $O(N)$.

Referensi:

- [1] <https://www.geeksforgeeks.org/analysis-algorithms-big-o-analysis/>
- [2] <https://www.tutorialspoint.com/time-and-space-complexity-in-data-str>
- [3] <https://www.geeksforgeeks.org/understanding-time-complexity-simple-examples/>

MODUL I : OVERVIEW OF C LANGUAGE

I. Tujuan

1. Melatih kemampuan untuk melakukan **dekomposisi masalah** dan memahami **struktur dasar**, serta **proses kompilasi** pada program C.
2. Memberikan pemahaman tentang deklarasi **variabel**, **tipe data dasar**, serta **input/output sederhana**.
3. Memberikan pemahaman tentang **operator dasar** dan struktur kontrol alur program (**kondisional dan perulangan**).
4. Memberikan pemahaman tentang penggunaan **fungsi dan rekursi** dalam program C.

II. Materi

II.1. Basic Structures

Proses kompilasi pada Bahasa C terbagi menjadi empat buah tahap seperti digambarkan pada Gambar 12. Masing-masing proses akan menerima masukan dan mengubahnya menjadi sebuah keluaran hingga akhirnya menjadi sebuah program yang dapat dieksekusi.



Gambar 12 Proses Kompilasi pada Bahasa C

Masing-masing proses memiliki kegunaannya masing-masing seperti dijelaskan berikut ini.

1. Pre-processing

Pada proses ini sebuah *file* dengan ekstensi *.c* akan diolah terlebih dahulu sebelum *compiler* melakukan tugasnya. Pada tahap ini akan menghasilkan sebuah *file* dengan ekstensi *.i*. Pengolahan yang dilakukan pada tahap ini meliputi :

- Menghapus komentar
- Menjabarkan definisi makro yang digunakan
- Menjabarkan definisi dari *file* yang di-*include*

Contohnya pada definisi `#define EULER 2.71828` akan diubah semua variabel `EULER` menjadi nilai yang sesuai. Definisi `#include <stdio.h>` akan diganti dengan isi dari *file* yang bersangkutan.

2. Compiling

Tahap ini akan melakukan kompilasi *file* dengan ekstensi `.i` menjadi *file assembly* dengan ekstensi `.s`. *File assembly* merupakan sebuah *file* yang berisi baris-baris instruksi yang dapat dipahami oleh mesin. Instruksi ini biasa dikenal sebagai *instruction set*.

3. Assembly

Pada tahap ini *file assembly* yang dihasilkan sebelumnya akan diubah menjadi sebuah *file* obyek dengan ekstensi `.o`. *File obyek* merupakan suatu *file* biner yang berisi instruksi yang dapat dipahami oleh mesin.

4. Linking

Tahap ini akan menggabungkan *file* obyek yang dihasilkan sebelumnya dengan definisi fungsi yang dipanggil pada program yang dibuat. Selain itu, pada tahap ini juga ditambahkan baris kode yang diperlukan untuk memulai dan mengakhiri sebuah program.

Terdapat berbagai jenis *compiler* yang dapat digunakan untuk melakukan kompilasi Bahasa C. Namun, pada praktikum ini akan digunakan GCC sebagai *compiler* standar. Untuk melakukan kompilasi Bahasa C dengan GCC dapat digunakan *syntax* berikut

<code>gcc -o <nama file output> <nama file source> -l<nama library></code>
--

Sebagai contoh apabila Anda memiliki *file* `main.c` sebagai program utama dan `lib.c` sebagai definisi fungsi serta dalam program Anda menggunakan *library* `math.h` maka Anda dapat melakukan kompilasi dengan menjalankan *syntax* berikut.

Pada Sistem Operasi Linux <code>gcc -o main main.c lib.c -lm</code>
Pada Sistem Operasi Windows <code>gcc -o main.exe main.c lib.c -lm</code>

Referensi:

[1] <https://www.geeksforgeeks.org/compiling-a-c-program-behind-the-scenes/>

II.II. Variable Declaration and Data Type

Dalam Bahasa C terdapat dua buah tipe data dasar yang dapat digunakan yakni tipe integer dan tipe *floating-point*. Masing-masing tipe dijabarkan lebih lanjut melalui tabel berikut.

Tabel 1 Tipe Data Integer

Tipe	Ukuran Data	Rentang Nilai
char	1 byte	-128 sampai 127 atau 0 sampai 255
unsigned char	1 byte	0 sampai 255
signed char	1 byte	-128 sampai 127
int	2 or 4 bytes	-32,768 sampai 32,767 atau -2,147,483,648 sampai 2,147,483,647
unsigned int	2 or 4 bytes	0 sampai 65,535 atau 0 sampai 4,294,967,295
short	2 bytes	-32,768 sampai 32,767
unsigned short	2 bytes	0 sampai 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 sampai 9223372036854775807
unsigned long	8 bytes	0 sampai 18446744073709551615

Tabel 2 Tipe Data Floating-Point

Tipe Data	Ukuran Data	Rentang Nilai	Presisi
float	4 byte	1.2×10^{-38} sampai 3.4×10^{38}	6 angka desimal
double	8 byte	2.3×10^{-308} sampai 1.7×10^{308}	15 angka desimal
long double	10 byte	3.4×10^{-4932} sampai 1.1×10^{4932}	19 angka desimal

Tipe data tersebut digunakan untuk mendefinisikan variabel yang akan digunakan dalam program. Proses pendefinisian variabel dapat menggunakan *syntax* berikut

```
type variable_list;
```

Dengan type merupakan tipe data yang valid dan variable_list merupakan nama variabel yang terdiri dari satu atau lebih variabel. Sebagai contoh, berikut merupakan pendefinisian beberapa jenis variabel. Apabila terdapat nama variabel lebih dari satu maka dapat menggunakan tanda koma sebagai pemisah.

```
int i, j, k;
char c, ch;
float f, salary;
double d;
```

Pendefinisian variabel juga dapat dilakukan bersamaan dengan memberikan nilai awal. Proses ini dapat dilakukan dengan menggunakan *syntax* berikut:

```
type variable_name = value;
```

Syntax di atas mirip dengan syntax sebelumnya hanya saja nilai awal langsung diberikan ketika mendefinisikan variabel. Contoh pendefinisian variabel dengan nilai awal dapat dilihat di bawah ini.

```
int d = 3, f = 5;
byte z = 22;
char x = 'x';
```

Referensi:

[1] https://www.tutorialspoint.com/cprogramming/c_data_types.htm

[2] https://www.tutorialspoint.com/cprogramming/c_variables.htm

II.III. Operator

Operator merupakan sebuah simbol yang berfungsi sebagai penanda bagi *compiler* untuk melakukan suatu operasi. Dalam Bahasa C terdapat beberapa jenis operator yaitu *Arithmetic Operators*, *Relational Operators*, *Logical Operators*, *Bitwise Operators*, dan *Assignment Operators*. Selain kelima jenis operator tersebut ada beberapa operator penting lainnya yang tidak masuk ke dalam lima kategori tersebut. Penjelasan dari masing-masing jenis operator akan dijabarkan melalui bagian berikut ini.

1. Arithmetic Operators

Arithmetic operators merupakan operator yang menandakan perintah untuk melakukan operasi matematika. Berikut penjelasan masing-masing operator yang masuk ke dalam kategori ini. Contoh berikut menggunakan nilai A yaitu 10 dan nilai B yaitu 20.

Tabel 3 Arithmetic Operators

Operator	Deskripsi	Contoh
+	Melakukan penjumlahan.	$A + B = 30$
-	Melakukan pengurangan.	$A - B = -10$
*	Melakukan perkalian.	$A * B = 200$
/	Melakukan pembagian.	$B / A = 2$
%	Mengambil nilai sisa hasil bagi (modulo).	$B \% A = 0$
++	Menambah nilai tipe data integer sebanyak 1.	$A++ = 11$
--	Mengurangi nilai tipe data integer sebanyak 1.	$A-- = 9$

2. Relational Operators

Relational operators merupakan simbol yang menandakan hubungan dari dua buah nilai yang dibandingkan. Berikut ditampilkan dalam tabel penjelasan dari masing-masing simbol yang terdapat pada jenis operator ini. Contoh di bawah menggunakan nilai A yaitu 10 dan nilai B yaitu 20.

Tabel 4 Relational Operators

Operator	Deskripsi	Contoh
==	Membandingkan kedua nilai apakah sama. Jika kedua nilai sama maka akan dihasilkan kondisi True.	$(A == B)$ is False.
!=	Membandingkan kedua nilai apakah berbeda. Jika kedua nilai berbeda maka akan dihasilkan kondisi True.	$(A != B)$ is True.
>	Membandingkan apakah nilai di sebelah kiri lebih besar dari nilai di sebelah kanan. Jika benar maka akan dihasilkan kondisi True.	$(A > B)$ is False.
<	Membandingkan apakah nilai di sebelah kiri lebih kecil dari nilai di sebelah kanan. Jika benar maka akan dihasilkan kondisi True.	$(A < B)$ is True.
>=	Membandingkan apakah nilai di sebelah kiri lebih besar atau sama dari nilai di sebelah kanan. Jika benar maka akan dihasilkan kondisi True.	$(A >= B)$ is False.
<=	Membandingkan apakah nilai di sebelah kiri lebih kecil atau sama dari nilai di sebelah kanan. Jika benar maka akan dihasilkan kondisi True.	$(A <= B)$ is True.

3. Logical Operators

Logical operators merupakan simbol yang menandakan perintah untuk melakukan operasi kondisi logika. Berikut ini dijelaskan simbol yang merupakan operator logika dalam Bahasa C. Contoh pada tabel menggunakan nilai A yaitu 1 dan nilai B yaitu 0.

Tabel 5 Logical Operators

Operator	Deskripsi	Contoh
&&	Operator logika AND. Ketika kedua kondisi bernilai True atau tidak 0 maka akan dihasilkan kondisi True.	(A && B) is False.
	Operator logika OR. Ketika salah satu dari kedua kondisi bernilai True atau tidak 0, maka akan dihasilkan kondisi True.	(A B) is True.
!	Operator logika NOT. Berguna untuk melakukan negasi dari suatu logika.	!(A && B) is True.

4. Bitwise Operators

Bitwise operators merupakan simbol untuk melakukan perintah operasi pada domain biner. Penjelasan untuk masing-masing simbol dijelaskan melalui tabel berikut. Contoh berikut menggunakan nilai A yaitu 60 (0011 1100) dan nilai B yaitu 13 (0000 1101).

Tabel 6 Bitwise Operators

Operator	Deskripsi	Contoh
&	Operator biner AND	(A & B) = 12, i.e., 0000 1100
	Operator biner OR	(A B) = 61, i.e., 0011 1101
^	Operator biner XOR	(A ^ B) = 49, i.e., 0011 0001
~	Operator biner NOT atau negasi	(~A) = ~(60), i.e., - 011 1100
<<	Operator biner LEFT SHIFT	A << 2 = 240 i.e., 1111 0000
>>	Operator biner RIGHT SHIFT	A >> 2 = 15 i.e., 0000 1111

5. Assignment Operators

Assignment operators merupakan simbol perintah untuk menyimpan dan/atau melakukan operasi. Penjelasan dari masing-masing simbol dapat diperhatikan pada tabel di bawah.

Tabel 7 Assignment Operators

Operator	Deskripsi	Contoh
=	Operator standar <i>assignment</i> . Menyimpan nilai dari operasi di sisi kanan ke variabel di sisi kiri.	$C = A + B$ menyimpan nilai $A + B$ ke dalam C
+=	Melakukan operasi penjumlahan antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C += A$ setara dengan $C = C + A$
-=	Melakukan operasi pengurangan antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C -= A$ setara dengan $C = C - A$
*=	Melakukan operasi perkalian antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C *= A$ setara dengan $C = C * A$
/=	Melakukan operasi pembagian antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C /= A$ setara dengan $C = C / A$
%=	Melakukan operasi modulo antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel	$C \% = A$ setara dengan $C = C \% A$
<<=	Melakukan operasi LEFT SHIFT antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C <<= 2$ setara dengan $C = C << 2$
>>=	Melakukan operasi RIGHT SHIFT antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C >>= 2$ setara dengan $C = C >> 2$
&=	Melakukan operasi biner AND antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C \&= 2$ setara dengan $C = C \& 2$
^=	Melakukan operasi biner XOR antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C \wedge= 2$ setara dengan $C = C \wedge 2$
=	Melakukan operasi biner OR antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri.	$C = 2$ setara dengan $C = C 2$

6. Operator Lainnya

Selain kelima jenis operator di atas terdapat beberapa operator lainnya yang penting dan umum digunakan dalam Bahasa C. Bagian ini membahas operator penting lainnya yang belum termasuk ke lima jenis di atas.

Tabel 8 Operator Penting Lainnya

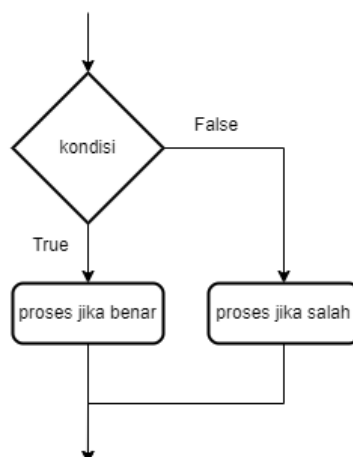
Operator	Deskripsi	Contoh
&	Me-return alamat memori dari variabel yang digunakan.	<code>scanf("%d", &i);</code> atau <code>mem = &a</code> , contoh pertama akan menyimpan nilai pada alamat variabel tersebut sedangkan contoh kedua artinya alamat memori dari variabel a disimpan pada variabel mem.
*	Pointer ke alamat memori dari sebuah variabel.	<code>*a</code> , merujuk pada data yang disimpan di memori a.

Referensi:

[1] https://www.tutorialspoint.com/cprogramming/c_operators.htm

II.IV. Branch

Struktur kondisional merupakan salah satu struktur yang sangat penting dalam pemrograman. Sebagian besar pola pada pemrograman akan menggunakan kondisional untuk menentukan pilihan yang mungkin terjadi dalam program tersebut. Kondisional akan memiliki struktur yang terdiri dari sebuah kondisi, operasi ketika kondisi benar, dan operasi ketika kondisi salah. Struktur tersebut dapat diterjemahkan menjadi *flowchart* berikut.



Gambar 13 Struktur Kondisional

Terdapat dua jenis kondisional yang terdapat dalam Bahasa C yaitu *if statement* dan *switch statement*. Masing-masing memiliki *syntax* yang berbeda. Penjelasan dari masing-masing jenis akan dijabarkan pada poin berikut ini.

1. If statement

Kondisional jenis ini merupakan jenis yang sering digunakan dan ditemukan dalam pemrograman. Secara umum *if statement* akan mengikuti struktur seperti Gambar 13 dengan *syntax* seperti berikut.

```
if(kondisi) {  
    kode jika benar  
}  
else {  
    kode jika salah  
}
```

Struktur di atas dapat diubah atau disesuaikan sesuai kebutuhan. Apabila hanya diperlukan kode yang benar saja maka kode dapat diubah menjadi seperti berikut.

```
if(kondisi) {  
    kode jika benar  
}
```

Struktur kondisional *if statement* juga dapat digunakan untuk lebih dari dua kondisi dengan menggunakan *syntax* berikut.

```
if(kondisi_1){  
    kode jika benar pertama  
}  
else if(kondisi_2){  
    kode jika benar kedua  
}  
else if ...  
  
else{  
    kode jika semua kondisi salah  
}
```

Selain variasi yang diberikan di atas, Anda juga dapat menggunakan *if statement* di dalam sebuah *if statement* lainnya. Struktur seperti ini biasa disebut dengan *nested if* dan memiliki *syntax* seperti berikut ini.

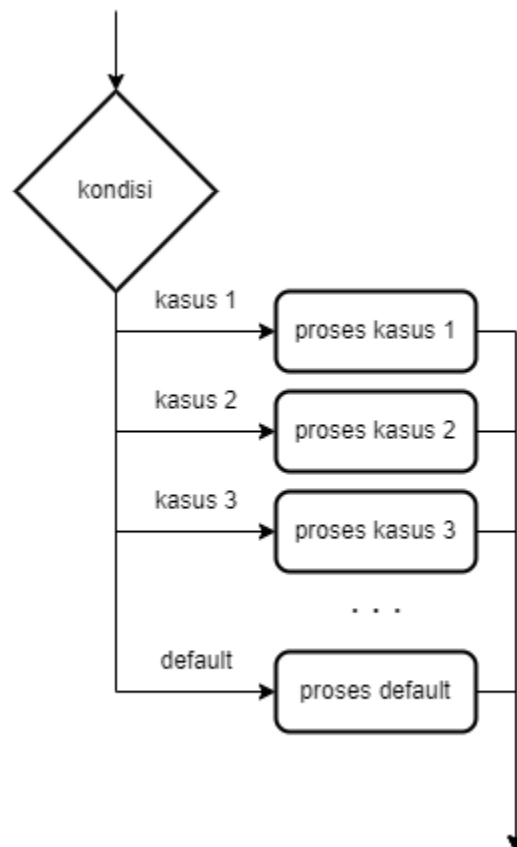
```

if(kondisi){
    if(kondisi_1){
        kode jika kondisi dan kondisi_1 benar
    }
    else{
        kode jika kondisi benar dan kondisi_1 salah
    }
}
else{
    if(kondisi_2){
        kode jika kondisi salah dan kondisi

```

2. Switch statement

Switch statement merupakan struktur yang biasa digunakan ketika terdapat lebih dari dua buah pilihan. Struktur ini tidak terlalu sering digunakan dalam program dibanding dengan *if statement*. Secara umum *switch statement* akan memiliki struktur seperti pada *flowchart* berikut.



Gambar 14 Struktur Switch Statement

Flowchart di atas menunjukkan bahwa struktur ini sangat cocok ketika pilihan yang tersedia ada banyak. *Switch statement* secara umum dapat dibentuk dengan menuliskan *syntax* berikut.

```
switch(variabel kondisi){
    case kondisi_1:
        kode kondisi_1
        break;
    case kondisi_2:
        kode kondisi_2
        break;
    ...
    default:
        kode default
}
```

Seperti pada *if statement*, *syntax switch statement* dapat diubah dan disesuaikan sesuai kebutuhan. *syntax* tersebut juga dapat diubah menjadi *nested switch*. Berikut merupakan *syntax* ketika menggunakan *nested switch*.

```
switch(kondisi_a){
    case kondisi_a_1:
        switch(kondisi_b){
            case kondisi_b_1:
                kode kondisi_b_1
                break;
            case kondisi_b_2:
                kode kondisi_b_2
                break;
            ...
            default:
                kode default
        }
        break;
    case kondisi_a_2:
        kode kondisi_2
        break;
    ...
    default:
        kode default
}
```

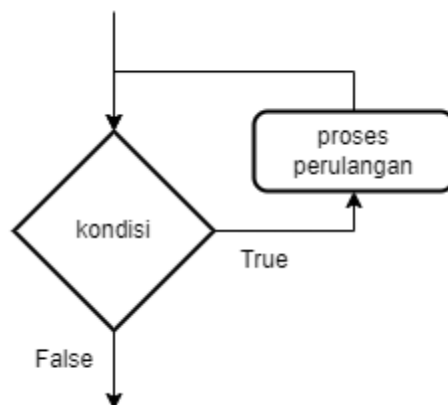
Referensi:

- [1] https://www.tutorialspoint.com/cprogramming/c_decision_making.htm
- [2] https://www.tutorialspoint.com/cprogramming/if_statement_in_c.htm
- [3] https://www.tutorialspoint.com/cprogramming/if_else_statement_in_c.htm
- [4] https://www.tutorialspoint.com/cprogramming/nested_if_statements_in_c.htm
- [5] https://www.tutorialspoint.com/cprogramming/switch_statement_in_c.htm

[6] https://www.tutorialspoint.com/cprogramming/nested_switch_statements_in_c.htm

II.V. Looping

Perulangan merupakan salah satu struktur yang sering digunakan dalam program selain kondisional. Sesuai namanya, struktur perulangan akan mengulangi bagian program tertentu hingga dihentikan oleh kondisi tertentu. Secara umum struktur perulangan akan mengikuti *flowchart* berikut.



Gambar 15 Struktur Perulangan

Dalam Bahasa C terdapat beberapa jenis perulangan yang dapat digunakan yaitu *while loop*, *for loop*, dan *do while loop*. Masing-masing jenis perulangan akan dijelaskan lebih lanjut pada poin berikut ini.

1. While loop

While loop akan melakukan perulangan secara terus menerus selama kondisi yang diberikan bernilai benar. Perulangan akan berhenti ketika kondisi yang diberikan bernilai salah. *While loop* akan memiliki *syntax* seperti berikut.

```
while(kondisi) {  
    kode perulangan  
}
```

2. For loop

For loop merupakan perulangan yang digunakan untuk mengulangi kode program apabila jumlah perulangannya sudah diketahui. Perulangan ini merupakan jenis yang sering digunakan dalam menyusun suatu program. *Syntax* struktur *for loop* dilihat di bawah ini.

```
for(inisialisasi; kondisi; increment){
    kode perulangan
}
```

Dari *syntax* di atas dapat dilihat bahwa terdapat tiga bagian yang menyusun *for loop* yaitu inisialisasi, kondisi, dan juga *increment*. Inisialisasi merupakan bagian untuk mendeklarasikan variabel kontrol dan nilainya. Bagian kondisi digunakan untuk mengevaluasi variabel kontrol apakah bernilai benar atau bernilai salah. *Increment* merupakan bagian untuk mengubah variabel kontrol yang digunakan sebelum dievaluasi kembali pada bagian kondisi.

3. Do while loop

Do while loop merupakan struktur perulangan yang mirip dengan *while loop*. Perbedaan dari *do while loop* dari *while loop* adalah proses evaluasi kondisi dilakukan di akhir setiap setelah kode perulangan dieksekusi. Apabila evaluasi kondisi bernilai benar maka kode perulangan akan dieksekusi kembali hingga evaluasi kondisi bernilai salah. *Syntax* berikut dapat digunakan untuk menyusun *do while loop*.

```
do{
    kode perulangan
}while (kondisi)
```

Referensi:

[1] https://www.tutorialspoint.com/cprogramming/c_loops.htm

[2] https://www.tutorialspoint.com/cprogramming/c_while_loop.htm

[3] https://www.tutorialspoint.com/cprogramming/c_for_loop.htm

[4] https://www.tutorialspoint.com/cprogramming/c_do_while_loop.htm

II.VI. Function

Fungsi merupakan sekelompok perintah yang memiliki tugas spesifik dalam bagian program. Fungsi berguna agar program lebih rapi dan mempermudah penulisan karena fungsi dapat digunakan secara terus menerus sesuai kebutuhan tanpa perlu menulis ulang.

Dalam membuat fungsi perlu diperhatikan bahwa terdapat deklarasi dan definisi fungsi. Deklarasi fungsi memuat informasi yaitu nama fungsi, tipe, dan parameter untuk digunakan oleh *compiler*. Definisi fungsi memuat keseluruhan fungsi termasuk perintah-perintah dalam fungsi tersebut. Secara umum fungsi dapat didefinisikan menggunakan format berikut.

```
return_type function_name( parameter list ) {  
    body of the function  
}
```

Dengan `return_type` merupakan tipe data yang dikembalikan oleh fungsi tersebut, `function_name` merupakan nama fungsi, parameter merupakan bagian yang digunakan untuk meneruskan nilai yang akan diproses oleh fungsi, dan *body* merupakan bagian fungsi yang berisi baris perintah.

Selain menggunakan definisi umum seperti atas, fungsi juga dapat dibuat dengan menggunakan deklarasi dan definisi secara terpisah. Deklarasi fungsi secara umum akan menggunakan format berikut

```
return_type function_name( parameter list ) {
```

Setelah membuat deklarasi selanjutnya dapat dilakukan definisi fungsi. Sebagai contoh diberikan fungsi untuk mengambil bilangan terbesar dari dua buah bilangan.

```
/* function declaration */  
int max(int num1, int num2);  
int main () {  
    /* local variable definition */  
    int a = 100;  
    int b = 200;  
    int ret;  
    /* calling a function to get max value */  
    ret = max(a, b);  
    printf( "Max value is : %d\n", ret );  
  
    return 0; }  
  
/* function definiton */  
/* function returning the max between two numbers */  
int max(int num1, int num2) {  
  
    /* local variable declaration */  
    int result;  
    if (num1 > num2)  
        result = num1;  
  
    else  
        result = num2;  
  
    return result;  
}
```

Perhatikan bahwa contoh di atas memiliki bagian deklarasi dan bagian definisi. Bagian deklarasi akan diletakkan pada bagian atas program sedangkan definisi diletakkan setelah bagian main. Selain metode tersebut, fungsi dapat dibuat dengan menuliskan definisi secara langsung pada bagian deklarasinya.

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_functions.htm

II.VII. Recursions

Fungsi rekursif merupakan fungsi yang memanggil dirinya sendiri di dalam prosesnya. Fungsi yang sudah dipelajari sebelumnya hanya memiliki proses yang sesuai dengan kebutuhan fungsi namun tidak memanggil dirinya sendiri. Fungsi rekursif dapat digunakan untuk menyelesaikan berbagai persoalan yang mudah maupun kompleks. Secara umum fungsi rekursif dapat dibentuk dengan *syntax* berikut.

```
void recursion() {  
    recursion(); /* function calls itself */  
}  
int main() {  
    recursion();  
}
```

Dalam menggunakan fungsi rekursif perlu diperhatikan dua hal penting yaitu kasus basis dan kasus rekursi agar tidak terjadi *infinite loop*. Kasus basis merupakan kasus paling dasar yang dapat menghentikan proses rekursi. Sedangkan kasus rekursi merupakan kondisi ketika program masih akan melakukan proses rekursi. Untuk memahami lebih lanjut silakan pelajari contoh dari referensi yang diberikan dan tutorial yang diberikan di bawah ini.

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_recursion.htm

III. Tutorial

Di bawah ini tersedia dua buah kode tutorial yang terkait dengan materi pada modul ini. Silakan Anda menyalin kode berikut dan menjalankannya pada komputer. Cermati proses eksekusi program sambil membaca dan memahami komentar serta maksud dari kode yang diberikan.

Source Code 3 Kode Tutorial 1 Modul 1

```
/**
 * EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul : 1 - Overview of C Language
 * Percobaan : Tutorial
 * Hari dan Tanggal : Jumat, 11 Februari 2026
 * Pembuat : Irfan Tito Kurniawan
 * Nama File : tutorial-m01-01.c
 * Deskripsi : Source code tutorial untuk modul 01.
 */

/**
 * Perintah Kompilasi
 *
 * Linux/MacOS:
 * gcc -o tutorial-m01-01 tutorial-m01-01.c -lm
 *
 * Windows:
 * gcc -o tutorial-m01-01.exe tutorial-m01-01.c
 *
 * atau dengan Makefile:
 * Linux/MacOS:
 * make 01
 */

/**
 * Catatan
 *
 * Kode ini dilengkapi dengan komentar rinci untuk membantu pemahaman.
 * Dalam praktiknya, hindari penggunaan komentar yang berlebihan.
 */

// Preprocessor Directives
#include <stdio.h>
#include <math.h>

// Macro definition
#define EULER 2.71828

int main() {
    // Deklarasi variabel
    int i;
    float f;
    char c;

    /** Blok input nilai variabel */
    printf("-- Fase Masukan\n");
    printf("Masukkan nilai bilangan bulat i: ");
    scanf("%d", &i);

    printf("Masukkan nilai karakter c: ");
    scanf(" %c", &c); // Spasi di depan %c untuk mengabaikan newline dari input sebelumnya

    printf("Masukkan nilai bilangan real f: ");
    scanf("%f", &f);

    /** Blok echo nilai variabel */
```

```

printf("\n-- Fase Keluaran\n");
printf("Nilai bilangan bulat i: %d\n", i);
printf("Nilai karakter c: %c\n", c);
printf("Nilai bilangan real f: %.3f\n", f);

/** Blok assignment dan ekspresi */
i = i * 2;
printf("Nilai variabel i setelah dikali 2: %d\n", i);

i = 5;
printf("Nilai variabel i setelah assignment: %d\n", i);

printf("Apakah nilai i kurang dari 5: %d\n", i < 5);
printf("Apakah nilai i sama dengan 5: %d\n", i == 5);

/** Blok perhitungan */
printf("\n-- Contoh Perhitungan\n");
printf("Nilai bilangan Euler: %.3f\n", EULER);
printf("Dengan math.h - Hasil dari e^(%d): %.3f\n", i, exp(i));
printf("Dengan macro - Hasil dari e^(%d): %.3f\n", i, pow(EULER, i));

// Return 0 mengindikasikan program selesai tanpa error
return 0;
}

```

MODUL II : DATA MANIPULATION AND EXTERNAL FILES

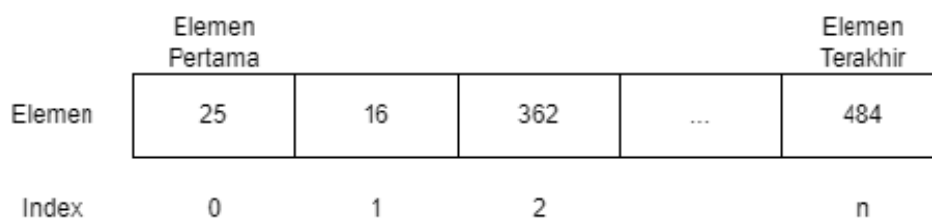
I. Tujuan

1. Memahami konsep **array** dan **pointer** serta cara penggunaannya dalam C.
2. Memberikan pemahaman tentang pengelolaan **string** dan cara kerja dengan **file eksternal** dalam C.
3. Memberikan pemahaman tentang cara **passing parameter** ke fungsi menggunakan passing by value dan passing by reference.

II. Materi

II.I. Array

Array adalah salah satu struktur data yang dapat menyimpan beberapa buah nilai dengan tipe data yang sama. Setiap nilai yang disimpan dalam sebuah *array* dapat diakses menggunakan *index*. Sebagai ilustrasi, gambar berikut menunjukkan bagaimana sebuah *array* menyimpan data.



Gambar 16 Ilustrasi Array

Dari gambar tersebut terlihat bahwa *array* akan memiliki banyak nilai/elemen sesuai dengan deklarasi yang diberikan. Masing-masing nilai akan menempati ruang memori secara terurut dari elemen pertama pada alamat memori yang paling kecil hingga elemen terakhir pada alamat memori yang paling besar. Masing-masing nilai dapat diakses menggunakan *index* yang berurut dari *index* 0 hingga *index* terakhir.

Deklarasi *array* dalam Bahasa C dapat dilakukan dengan mengikuti *syntax* berikut

```
type arrayName [ arraySize ] ;
```

Dengan *type* merupakan tipe data dari *array*, *arrayName* merupakan variabel *array* yang digunakan, dan *arraySize* merupakan ukuran *array* yang dibuat. Seperti deklarasi variabel, deklarasi *array* juga dapat langsung diinisialisasi dengan menggunakan *syntax* berikut.

```
type arrayName [ arraySize ] = {value_1, value_2, ..., value_n};
```

Perlu diperhatikan bahwa banyaknya nilai yang diinisialisasi tidak boleh lebih dari ukuran *array*. Contoh terkait pendefinisian *array* dapat Anda lihat pada kode berikut.

```
double balance[5];  
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

Setelah mempelajari proses mendefinisikan *array* maka berikutnya akan dipelajari proses untuk mengakses *array*. Sebuah *array* dapat diakses elemennya dengan menggunakan *index* yang sesuai. Seperti diilustrasikan pada Gambar 16, setiap elemen memiliki *index* yang sesuai. *Syntax* yang digunakan dalam mengakses *array* adalah sebagai berikut.

```
arrayName [ index ]
```

Dengan menggunakan *syntax* tersebut kita dapat mengakses salah satu elemen pada *array* dan melakukan operasi. Operasi tersebut dapat berupa menyimpan nilai atau mengambil nilai. Sebagai contoh, terdapat *array* *balance* dengan isi seperti contoh sebelumnya. *Array* tersebut dapat diambil nilainya dan diubah nilainya.

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};  
double number = balance[3] //number bernilai 7.0  
balance[2] = 4.8 //balance berisi {1000.0, 2.0, 4.8, 7.0, 50.0}
```

Dari contoh di atas jelas bahwa dengan mengakses *index* dari sebuah *array* dapat dilakukan manipulasi elemen di dalamnya. Perlu diperhatikan bahwa angka di dalam kurung siku pada baris pertama dan dua baris lainnya merupakan hal yang berbeda. Pada baris pertama, yang merupakan inisialisasi, merupakan ukuran dari *array*, sedangkan pada baris kedua dan ketiga merupakan *index* dari elemen yang ingin diakses. Anda dapat menggunakan konsep perulangan untuk mengakses *array* secara lebih mudah ketimbang menuliskan *index*-nya secara manual.

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_arrays.htm

II.II. *String*

String dalam Bahasa C merupakan sebuah *array* dengan tipe data *char*, artinya *string* merupakan kumpulan dari beberapa karakter. *String* dalam Bahasa C akan selalu diakhiri dengan sebuah *null* karakter ‘\0’. Untuk mendeklarasikan sebuah *string* dapat digunakan cara

yang sama seperti saat mendeklarasikan *array*. Berikut diberikan dua contoh cara mendeklarasikan *string* dalam Bahasa C.

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
char greeting[] = "Hello";
```

Anda dapat mendeklarasikan ukuran *string* yang akan digunakan kemudian memasukkan karakter yang sesuai satu per satu. Selain itu, Anda juga bisa menggunakan cara tanpa memberikan ukuran *string* lalu langsung menuliskan kalimat yang diperlukan dan biarkan *compiler* yang mengatur ukurannya. Karakter *null* pada bagian akhir contoh baris satu sesungguhnya tidak perlu ditulis secara eksplisit karena *compiler* secara otomatis akan menambahkannya.

Pada umumnya untuk melakukan operasi pada *string* diperlukan sebuah *library* tambahan untuk mempermudah yaitu *string.h*. Dalam *library* tersebut terdapat berbagai macam fungsi yang dapat digunakan untuk memanipulasi *string*. Beberapa fungsi yang biasa digunakan ditampilkan pada tabel berikut.

Tabel 9 Contoh Fungsi dalam Library String.h

No.	Fungsi	Deskripsi
1	<code>char *strtok(char *str, const char *delim)</code>	Digunakan untuk memisahkan sebuah <i>string</i> menjadi beberapa buah <i>string</i> yang dipisahkan dengan <i>delimiter</i> tertentu.
2	<code>char *strcpy(char *dest, const char *src)</code>	Digunakan untuk menyalin sebuah <i>string</i> .
3	<code>int strcmp(const char *str1, const char *str2)</code>	Digunakan untuk membandingkan dua buah <i>string</i> apakah sama atau tidak.
4	<code>size_t strlen(const char *str)</code>	Digunakan untuk menghitung panjang dari suatu <i>string</i> tanpa melibatkan <i>null</i> karakter.
5	<code>char *strcat(char *dest, const char *src)</code>	Digunakan untuk menambahkan suatu <i>string</i> ke akhir dari <i>string</i> lainnya.

Tabel di atas memberikan beberapa contoh fungsi yang umum digunakan. Anda dapat membaca lebih lanjut fungsi-fungsi lainnya yang terdapat pada *library* string.h pada referensi di bawah. Pada referensi tersebut juga tersedia contoh penggunaan dari masing-masing fungsi.

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_strings.htm

[2] https://www.tutorialspoint.com/c_standard_library/string_h.htm

II.III. *Pointer*

Seperti yang sudah dibahas pada bagian sebelumnya, variabel merupakan sebuah alokasi memori yang punya alamat dan data yang disimpan di dalamnya. Alamat memori dari sebuah variabel dapat diakses menggunakan *&*. *Pointer* merupakan variabel yang menyimpan alamat memori dari variabel lainnya. Sama seperti variabel pada umumnya, *pointer* perlu dideklarasikan terlebih dahulu sebelum digunakan. *Syntax* yang digunakan untuk mendeklarasikan *pointer* adalah sebagai berikut,

```
type *var-name;
```

Bagian *type* merupakan tipe data dari variabel yang akan kita simpan alamat memorinya sedangkan *var-name* merupakan nama variabel *pointer* yang digunakan. Berikut diberikan beberapa contoh deklarasi *pointer*.

```
int *ip; /* pointer to an integer */
double *dp; /* pointer to a double */
float *fp; /* pointer to a float */
char *ch /* pointer to a character */
```

Setelah mengetahui proses deklarasi dari *pointer* maka selanjutnya akan dibahas cara menggunakan *pointer*. Terdapat tiga buah tahap yang diperlukan dalam menggunakan *pointer* yaitu mendeklarasikan *pointer*, menyimpan alamat dari suatu variabel pada *pointer*, dan mengakses nilai dari alamat yang disimpan pada *pointer*. Proses menyimpan alamat memori dapat menggunakan operator *&* sedangkan untuk mengakses nilai dari alamat memori pada sebuah *pointer* dapat menggunakan operator ***. Berikut diberikan contoh penggunaan operator tersebut.

```
/*assume address of var is 0x08bfacdf*/
int var = 20; /* actual variable declaration */
int *ip; /* pointer variable declaration */

ip = &var; /* store address of var in pointer variable*/
printf("Address of var variable: %p\n", &var );
```

```
/* address stored in pointer variable */
printf("Address stored in ip variable: %p\n", ip );

/* access the value using the pointer */
printf("Value of *ip variable: %d\n", *ip );
```

Asumsikan variabel *var* tersimpan pada alamat memori di atas, maka pada baris ke tiga alamat memori dari *var* akan disimpan pada *pointer* *ip*. Setelah alamat memori disimpan pada *pointer* maka selanjutnya *pointer* tersebut dapat digunakan untuk operasi. Sebagai contoh dilakukan operasi untuk mencetak pada layar, dua perintah cetak pertama akan menghasilkan nilai yang sama yaitu alamat memori dari *var* yakni 0x08bfacdf karena pada baris sebelumnya sudah dilakukan proses penyimpanan alamat memori pada *pointer*. Perintah cetak pada baris terakhir akan menghasilkan nilai yang disimpan oleh variabel *var* yaitu 20. Perhatikan ketika mengakses nilai yang disimpan pada variabel maka digunakan operator ***.

Selain penjelasan di atas, terdapat aplikasi dari *pointer* yang lebih dalam. Penjelasan lebih lanjut terkait *pointer* dapat dibaca pada referensi yang diberikan. Silakan lakukan eksplorasi secara mandiri.

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_pointers.htm

II.IV. File Eksternal

Dalam Bahasa C tersedia fitur untuk mengolah sebuah *file* eksternal. Proses pengolahan meliputi membuka *file*, menulis *file*, membaca *file*, dan menutup *file*. Masing-masing operasi tersebut akan dibahas lebih detail pada bagian berikutnya.

1. Membuka File

Proses membuka suatu *file* eksternal dapat dilakukan dengan menggunakan fungsi *fopen()*. Fungsi tersebut akan menerima parameter berupa nama *file* dan mode yang digunakan. Untuk lebih jelasnya berikut adalah *syntax* yang digunakan.

```
FILE *fopen( const char * filename, const char * mode );
```

Dalam Bahasa C terdapat beberapa mode pembukaan yang dapat digunakan. Penjelasan untuk masing-masing mode ditampilkan pada tabel berikut.

Tabel 10 Mode Pembacaan File Eksternal dalam Bahasa C

No.	Mode	Deskripsi
1.	r	Membuka <i>file</i> yang sudah ada untuk dibaca isinya.
2.	w	Membuka <i>file</i> untuk dituliskan isinya. Jika <i>file</i> belum ada maka akan dibuat sebuah <i>file</i> baru.
3.	a	Membuka <i>file</i> untuk dituliskan isinya secara <i>appending</i> . Jika <i>file</i> belum ada maka akan dibuat <i>file</i> baru.
4.	r+	Membuka <i>file</i> untuk membaca dan menulis isinya.
5.	w+	Membuka <i>file</i> untuk membaca dan menulis isinya. Jika <i>file</i> sudah ada maka <i>file</i> akan dipotong hingga panjangnya nol. Jika <i>file</i> tidak ada maka akan dibuat sebuah <i>file</i> baru.
6.	a+	Membuka <i>file</i> untuk membaca dan menulis isinya. Jika <i>file</i> tidak ada maka akan dibuat sebuah <i>file</i> baru. Proses membaca bisa dilakukan dari awal <i>file</i> , namun proses menulis hanya bisa dilakukan secara <i>appending</i> .

2. Menulis File

Proses menulis *file* dapat dilakukan menggunakan dua cara yaitu satu persatu tiap karakter atau secara langsung dalam bentuk *string*. *Syntax* untuk masing-masing metode tersebut dapat dilihat pada tabel berikut.

```
int fputc( int c, FILE *fp );
int fputs( const char *s, FILE *fp );
```

Kedua metode tersebut akan menerima dua buah parameter yaitu data yang akan ditulis dalam bentuk karakter atau *string* dan *file* yang akan digunakan untuk menulis.

3. Membaca File

Mirip seperti proses menulis, pada proses membaca *file* juga terdapat dua buah metode yang dapat digunakan yaitu membaca karakter dan membaca satu baris *string*. *Syntax* yang digunakan untuk melakukan pembacaan *file* dapat dilihat pada tabel berikut.

```
int fgetc( FILE * fp );
char *fgets( char *buf, int n, FILE *fp );
```

Pembacaan *file* berdasarkan karakter, *syntax* baris pertama, hanya memerlukan sebuah parameter yaitu *file* yang akan dibaca. Berbeda dengan metode pembacaan karakter, pembacaan berdasarkan *string* memerlukan tiga buah parameter yaitu variabel untuk menyimpan *string* yang dibaca, panjang *string* yang bisa dibaca, dan nama *file* yang dibaca.

4. Membaca File

Proses menutup suatu *file* dapat dilakukan dengan mudah menggunakan fungsi `fclose()`. Fungsi ini akan memiliki *syntax* sebagai berikut.

```
int fgetc( FILE * fp );
```

Fungsi tersebut akan memberikan nilai **EOF** jika terjadi eror saat menutup sebuah *file*.

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_file_io.htm

II.V. *Passing Parameter*

Passing parameter adalah proses mengirimkan data dari satu bagian program ke bagian lain, biasanya saat memanggil sebuah fungsi atau metode. Parameter digunakan untuk memberikan informasi yang diperlukan fungsi tersebut untuk bekerja.

1. *Passing by Value*

Passing by value merupakan pengiriman parameter yang dilakukan dengan meng-copy-kan value dari actual parameter (dari pemanggil/*caller*) ke formal parameter (function). Perubahan yang terjadi di dalam function tidak akan mempengaruhi nilai asli pemanggilnya.

Perhatikan contoh berikut ini untuk lebih jelasnya.

```
#include<stdio.h>
void swap (int a, int b) {
    int c = a;
    a = b;
    b =
}
int main() {
    int x = 7, y = 10;
    swap(x,y);
```

```
printf("x= %d dan y %d\n", x, y); =  
return 0;  
}
```

Pada contoh di atas ini, dibuat sebuah fungsi **swap** yang menerima 2 parameter yakni int a, dan int b, dan bertujuan untuk menukar isi dari a dan b. Bila kita jalankan program ini, maka output program adalah: **“x = 7 dan y = 10”**. Hal ini terjadi dikarenakan perubahan nilai hanya terjadi di dalam fungsi swap. Dimana bila kita mencetak data a dan b di sana, nilai memang benar sudah terganti. Namun dikarenakan kita mengirimkan copy value dari nilai tersebut, maka tidak mempengaruhi x dan y dari sisi pemanggil.

2. *Passing by Reference*

Passing by reference adalah metode pengiriman parameter ke sebuah fungsi atau metode di mana parameter formal menerima referensi (alamat memori) dari parameter aktual, bukan salinan nilai. Ini berarti perubahan yang dilakukan pada parameter formal dalam fungsi akan langsung memengaruhi parameter aktual di luar fungsi.

Dalam passing by reference, actual parameter yang kita kirimkan adalah alamat memori (memory address dari variable yang diinginkan), dan formal parameternya merupakan pointer yang menunjuk ke alamat memori tersebut. Dengan demikian perubahan yang dilakukan akan mempengaruhi nilai pada alamat yang ditunjuk (yakni alamat x dan y yang merupakan pemanggilnya). Dengan demikian bila program di atas dieksekusi, output yang akan dihasilkan adalah **“x = 10 dan y = 7”**.

```
#include<stdio.h>  
void swap (int* a, int* b) {  
    int c = *a;  
    *a = *b;  
    *b = c;  
}  
int main() {  
    int x = 7, y = 10;  
    swap(&x,&y);  
    printf("x= %d dan y %d\n", x, y);  
    return 0;  
}
```

Referensi

[1] <https://www.geeksforgeeks.org/g-fact-6/>

[2] <https://perfectelearning.com/blog/c-programming-technical-interview-questions-for>

III. Tutorial

Di bawah ini tersedia dua buah kode tutorial yang terkait dengan materi pada modul ini. Silakan Anda menyalin kode berikut dan menjalankannya pada komputer. Cermati proses eksekusi program sambil membaca dan memahami komentar serta maksud dari kode yang diberikan.

Source Code 4 Kode Tutorial 1 Modul 2

```
/**
 * EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul: 2 - Pointers
 * Percobaan: Tutorial
 * Hari dan Tanggal: Selasa, 3 Januari 2026
 * Nama File: tutorial-03.c
 * Pembuat: Stefen Sutandi
 * Deskripsi: Source code tutorial untuk modul 02.
 */

/**
 * Perintah Kompilasi
 *
 * Linux/MacOS - gcc -o tutorial-03 tutorial-03.c
 * Windows - gcc -o tutorial-03.exe tutorial-03.c
 *
 * atau dengan Makefile:
 *
 * Linux/MacOS - make 01
 */

#include <stdio.h>
#include <stdlib.h>

/** Fungsi dengan definisi umum */
int add_int(int operand_1, int operand_2) {
    return operand_1 + operand_2;
}

/** Fungsi dengan deklarasi */
void say_hi();

int main() {
    /** Bagian ini menjelaskan pointer */
    // Deklarasi variabel
    int number = 24;
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr_to_number;

    // Simpan alamat variabel number pada pointer ptr_to_number
    ptr_to_number = &number;

    // Cetak nilai alamat dan value
    printf("Nilai pada variabel number: %d\n", number);
    printf("Alamat pada variabel number: %p\n", &number);
    printf("Nilai pada pointer ptr_to_number: %p\n", ptr_to_number);
    printf("Nilai yang dirujuk pointer ptr_to_number: %d\n", *ptr_to_number);
    printf("\n");
}
```

```

// Pointer dan array
// Memanggil nama array saja (tanpa index) akan merujuk ke alamat pertama dari
array tersebut
printf("Alamat pertama dari sebuah array: %p\n", arr);
printf("Elemen pertama dari sebuah array: %d\n", arr[0]);

// Elemen dari array dapat dipanggil dengan menggunakan metode yang sama dengan
memanggil nilai yang dirujuk pointer
printf("Elemen pertama dari sebuah array: %d\n", *arr);
printf("Elemen kedua dari sebuah array: %d\n", *(arr + 1));
printf("Elemen ketiga dari sebuah array: %d\n", *(arr + 2));
printf("Elemen keempat dari sebuah array: %d\n", *(arr + 3));
printf("Elemen kelima dari sebuah array: %d\n", *(arr + 4));
printf("\n");

/** Bagian ini menjelaskan function */
// Functions tipe Void
say_hi();
printf("\n");

// Deklarasi Variabel
int a = 31;
int b = 14;

// Functions tipe Int
printf("Hasil penjumlahan dari variabel: %d (31+14)\n", add_int(a, b));
printf("Hasil penjumlahan dari operand langsung: %d (12+8)\n", add_int(12, 8));

return 0;
}

/** Definisi dari fungsi sebelumnya */
void say_hi() {
    printf("Hello, Konichiwa, Bonjour\n");
}

```

Source Code 5 Kode Tutorial 2 Modul 2

```

/**
 * EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul : 2 - Strings and External Files
 * Percobaan : Tutorial
 * Hari dan Tanggal : Jumat, 18 Februari 2026
 * Nama File : tutorial-02.c
 * Pembuat : Stefen Sutandi
 * Deskripsi : Source code tutorial untuk modul 02.
 */

/** Perintah Kompilasi
 * Linux/MacOS: gcc -o tutorial-02 tutorial-02.c
 * Windows: gcc -o tutorial-02.exe tutorial-02.c
 * atau dengan Makefile:
 * Linux/MacOS: make 01
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Panjang maksimum entry
define MAX_LEN 255

int main() {
    /** Print 'string' yang konstan.
     * Catatan: lebihkan ukuran variabel karena string

```

```

    * diakhiri dengan karakter terminator '\0'.
    */
const char dummy[7] = "Henlo!";
printf("Test print string: %s\n", dummy);

// Input nama file
char filename[MAX_LEN];
printf("\nMasukkan nama file: ");
scanf("%s", filename);

// Buka file untuk dibaca
FILE* stream = fopen(filename, "r");
if (stream == NULL) {
    perror("Gagal membuka file");
    return 1;
}

// Array nama mahasiswa
char names[MAX_LEN][MAX_LEN];
// Array nilai mahasiswa
float score[MAX_LEN];

// Variabel penyimpan baris dari file sementara
char line[MAX_LEN];
// Token penyimpan string sementara untuk parsing
char* token;
// Iterator penanda indeks array, jumlah baris pada file
int i = 0;

// Baca file baris demi baris hingga habis
while (fgets(line, MAX_LEN, stream)) {
    // Parse baris
    // Ambil string pada baris hingga tanda koma (koma tidak ikut dibaca)
    token = strtok(line, ",");
    // Copy string dari token ke array nama
    strcpy(names[i], token);
    // Ambil string pada baris hingga karakter newline (newline tidak ikut dibaca)
    token = strtok(NULL, "\n");
    // Convert string yang dibaca menjadi float dengan atof
    score[i] = atof(token);
    ++i;
}

// Cari nilai maksimum, minimum, dan rata-rata nilai
float max = score[0];
float min = score[0];
float sum = score[0];
int max_idx = 0;
int min_idx = 0;

for (int j = 1; j < i; ++j) {
    // Cari nilai maksimum dan minimum
    if (score[j] > max) {
        max = score[j];
        max_idx = j;
    }
    if (score[j] < min) {
        min = score[j];
        min_idx = j;
    }
    // Jumlahkan nilai untuk menghitung rata-rata
    sum += score[j];
}

// Nilai rata-rata
float mean = sum / i;

```

```

// Print semua nama dan nilai
for (int j = 0; j < i; ++j) {
    printf("%s memperoleh nilai %.2f\n", names[j], score[j]);
}

// Print statistik
printf("\nStatistik:\n");
printf("Nilai tertinggi: %.2f, atas nama %s\n", max, names[max_idx]);
printf("Nilai terendah: %.2f, atas nama %s\n", min, names[min_idx]);
printf("Nilai rata-rata: %.2f\n", mean);

// Nama sebagai kunci pencarian
char key[MAX_LEN];
// Input kunci pencarian
printf("\nCari nilai atas nama: ");
scanf("%s", key);

// Hitung panjang nama dengan strlen dan tampilkan sebagai long int
printf("%s terdiri atas %ld karakter.\n", key, strlen(key));

// Cari nama hingga ditemukan atau hingga array habis
int found = 0;
int j = 0;
while (j < i && !found) {
    if (!strcmp(key, names[j])) {
        found = 1;
    } else {
        ++j;
    }
}

// Print nilai kunci pencarian bila ditemukan
if (found) {
    printf("%s memperoleh nilai: %.2f\n", key, score[j]);
} else {
    printf("%s tidak ada.\n", key);
}

// Tutup file
fclose(stream);
return 0;
}

```

Tabel 11 File Eksternal Tutorial Modul 2

Andi,70.5 Budi,40.5 Caca,20.9 Dedi,80.9 Eko,95.4
--

MODUL III : STRUCTURES AND DYNAMIC ARRAYS

I. Tujuan

1. Mengembangkan keterampilan **dekomposisi masalah** untuk merancang solusi yang lebih terstruktur dan efisien.
2. Memberikan pemahaman yang komprehensif mengenai **tipe data bentukan** dalam Bahasa C dan penerapannya dalam pemrograman.
3. Meningkatkan pemahaman praktikan terkait penggunaan **array dinamis** dalam Bahasa C, serta cara mengelola memori secara efisien dengan pendekatan tersebut.

II. Materi

II.1. Array Dinamis

Memory management atau sering disebut sebagai *array* dinamis merupakan metode dalam Bahasa C yang digunakan untuk mengatur ukuran dari sebuah *array*. Ukuran *array* pada umumnya ditulis secara eksplisit pada kode program dan tidak dapat diubah selama program berjalan. *Array* dinamis memberikan kemudahan untuk kondisi ketika ukuran *array* yang akan digunakan belum diketahui secara pasti. *Array* dinamis memungkinkan perubahan ukuran *array* ketika program sedang dijalankan.

Array dinamis dapat dijalankan dengan menggunakan fungsi yang terdapat dalam *library* `stdlib.h`. Terdapat empat buah fungsi yang secara umum digunakan untuk mengelola *array* dinamis. Masing-masing fungsi dijelaskan pada tabel berikut.

Tabel 12 Fungsi Pengolahan Array Dinamis dalam Bahasa C

No.	Fungsi	Deskripsi
1.	<code>void *calloc(int num, int size)</code>	Fungsi ini akan membuat <i>array</i> dengan ukuran sesuai parameternya. <i>Array</i> yang dibuat akan memiliki nilai awal yaitu 0.
2.	<code>void free(void *address)</code>	Fungsi ini digunakan untuk menghapus blok memori / <i>array</i> dengan alamat tertentu.
3.	<code>void *malloc(size_t size)</code>	Fungsi ini akan membuat <i>array</i> baru dengan ukuran tertentu.

4.	<code>void *realloc(void *address, int newsize)</code>	Fungsi ini digunakan untuk mengubah ukuran <i>array</i> yang sudah dibuat sebelumnya.
----	--	---

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_memory_management.htm

[2] https://www.tutorialspoint.com/c_standard_library/stdlib_h.htm

II.II. Structure

Sebelumnya sudah diajarkan bahwa *array* dapat menyimpan banyak data dengan tipe yang sama. Mirip dengan *array*, *structure* juga dapat menyimpan banyak data hanya saja tipe data yang dapat disimpan dapat berbeda-beda. *Structure* bekerja dengan cara mendefinisikan tipe data baru yang berisikan tipe data dasar. Untuk mendefinisikan *structure* dapat digunakan *syntax* berikut.

```
struct [structure tag] {
    member definition;
    member definition;
    ...
    member definition;
} [one or more structure variables];
```

Dengan structure tag merupakan nama dari tipe data baru yang dibentuk, member definition merupakan deklarasi variabel, dan structure variable merupakan nama variabel yang dibentuk dengan tipe data bentukan tersebut (opsional). Sebagai contoh, diberikan *structure* untuk mendefinisikan tipe data buku.

```
struct Books {
    char title[50];
    char author[50];
    char subject[100];
    int book_id;
} book;
```

Contoh di atas dapat diartikan bahwa terdapat tipe data bentukan baru yang bernama Books dengan komponen penyusun yaitu title, author, subject, dan book_id. Terdapat juga sebuah

variabel dengan tipe bentukan yang bernama book. Perhatikan bahwa komponen penyusun dari tipe data bentukan dapat berupa *array* maupun tipe data dasar. Dari contoh di atas terlihat bahwa sudah terbentuk sebuah variabel bernama book yang memiliki tipe data Books. Untuk mendeklarasikan tipe data bentukan dapat dilakukan seperti variabel biasa yakni mengikuti *syntax* berikut.

```
struct [structure tag] nama_variabel
//contoh
struct Books Book1;
```

Komponen dari sebuah *structure* dapat diakses masing-masing menggunakan tanda titik (.). Menggunakan contoh di atas komponen dari tipe data Books dapat diakses menggunakan *syntax* berikut.

```
strcpy( book.title, "C Programming");
strcpy( book.author, "Nuha Ali");
strcpy( book.subject, "C Programming Tutorial");
book.book_id = 6495407;
//atau
strcpy( Book1.title, "C Programming");
strcpy( Book1.author, "Nuha Ali");
strcpy( Book1.subject, "C Programming Tutorial");
Book1.book_id = 6495407;
```

Perhatikan bahwa diperlukan nama dari variabel yang digunakan yaitu book atau Book1 kemudian diikuti dengan nama komponen yang ingin diakses. Sebuah *structure* dapat diperlakukan seperti variabel pada umumnya. *Structure* dapat dijadikan sebuah *pointer* dan bisa dijadikan sebagai parameter sebuah fungsi. Penggunaan *structure* sebagai sebuah *pointer* tetap menggunakan tanda (*) untuk deklarasi. Untuk mengakses elemennya menggunakan tanda (->). Perhatikan bahwa untuk mengakses elemen *structure* biasa dengan *structure pointer* memiliki cara yang berbeda. Perhatikan contoh berikut untuk melakukan deklarasi dan mengakses *structure pointer*.

```
struct Books *struct_pointer;
struct_pointer = &Book1;
printf( "Book title : %s\n", book->title);
printf( "Book author : %s\n", book->author);
printf( "Book subject : %s\n", book->subject);
```

```
printf( "Book book_id : %d\n", book->book_id);
```

Referensi

[1] https://www.tutorialspoint.com/cprogramming/c_structures.htm

III. Tutorial

Di bawah ini tersedia kode tutorial yang terkait dengan materi pada modul ini. Silakan Anda menyalin kode berikut dan menjalankannya pada komputer. Cermati proses eksekusi program sambil membaca dan memahami komentar serta maksud dari kode yang diberikan.

Source Code 6 Kode Tutorial 1 Modul 3

```
/** EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul : 3 - Structures and Dynamic Arrays
 * Percobaan : Tutorial
 * Hari dan Tanggal : Jumat, 12 Maret 2026
 * Nama File : tutorial-03-arr.c
 * Pembuat : Stefen Sutandi
 * Deskripsi : Source code tutorial untuk modul 03, materi array dinamis.
 */

/** Perintah Kompilasi
 *
 * Linux/MacOS - gcc -o tutorial-04-arr tutorial-04-arr.c
 * Windows - gcc -o tutorial-04-arr.exe tutorial-04-arr.c
 *
 * atau dengan Makefile:
 *
 * Linux/MacOS - make arr
 */

#include <stdio.h>
#include <stdlib.h>

// Jumlahkan isi array dinamis arr dengan ukuran arr_size.
int sum_arr(int* arr, int arr_size) {
    int sum = 0;
    for (int i = 0; i < arr_size; ++i) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    // Deklarasi array dinamis
    // Pointer to int yang akan menyimpan address elemen pertama array
    int* arr = NULL;
    // Ukuran array dinamis sejauh ini
    int arr_size = 0;
    // Variabel penyimpanan input sementara sebelum disimpan dalam array
    int temp;

    // Input array secara dinamis hingga input -1
    printf("Input elemen array (-1 untuk selesai): ");
    scanf("%d", &temp);

    // Loop selama input tidak sama dengan -1
    while (temp != -1) {
        // Catat penambahan ukuran array
```

```

        ++arr_size;

        // Alokasi memori tambahan untuk menampung input
        if (arr_size == 1) {
            // Bila array belum diinisialisasi
            // Reservasi memori sebesar 1 buah integer dengan malloc
            arr = (int*) malloc(sizeof(int));
        } else {
            // Bila array sudah diinisialisasi
            // Alokasi ulang dengan ukuran sebesar arr_size buah integer dengan
realloc
            arr = (int*) realloc(arr, arr_size * sizeof(int));
        }

        // Assign nilai elemen terakhir array dengan input
        arr[arr_size - 1] = temp;

        // Input elemen berikutnya
        printf("Input elemen array (-1 untuk selesai): ");
        scanf("%d", &temp);
    }

    // Print seluruh isi array
    printf("\nIsi array:\n");
    for (int i = 0; i < arr_size; ++i) {
        printf("%d\n", arr[i]);
    }

    // Print jumlah isi array arr
    printf("Jumlah isi array: %d\n", sum_arr(arr, arr_size));

    // Reset isi array
    // Free memori yang digunakan
    free(arr);
    // Set ukuran array ke nol kembali
    arr_size = 0;

    // Input ukuran array
    printf("\nJumlah elemen array: ");
    scanf("%d", &arr_size);

    // Alokasi memori dan inisialisasi array sebesar arr_size buah integer
    arr = (int*) calloc(arr_size, sizeof(int));

    // Print seluruh isi array sebelum input
    printf("\nIsi array sebelum input:\n");
    for (int i = 0; i < arr_size; ++i) {
        printf("%d\n", arr[i]);
    }

    // Input isi array
    printf("\nInput isi array:\n");
    for (int i = 0; i < arr_size; ++i) {
        printf("Input elemen ke-%d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    // Print seluruh isi array
    printf("\nIsi array setelah input:\n");
    for (int i = 0; i < arr_size; ++i) {
        printf("%d\n", arr[i]);
    }

    // Free memori yang digunakan
    free(arr);
    return 0;
}

```

MODUL IV : DYNAMIC STRUCTURES

I. Tujuan

1. Meningkatkan keterampilan dalam **dekomposisi masalah** untuk merancang solusi yang lebih efektif dan efisien.
2. Memberikan wawasan mendalam mengenai **struktur data linked list** dalam Bahasa C, serta penerapannya dalam berbagai konteks pemrograman.
3. Mengembangkan pemahaman praktikan terkait **struktur data stack** dan **queue**, serta teknik-teknik pengelolaan data yang efisien.
4. Membekali praktikan dengan pemahaman tentang **algoritma pencarian** dan **pengurutan** yang dioptimalkan untuk struktur data berbasis linked structure.

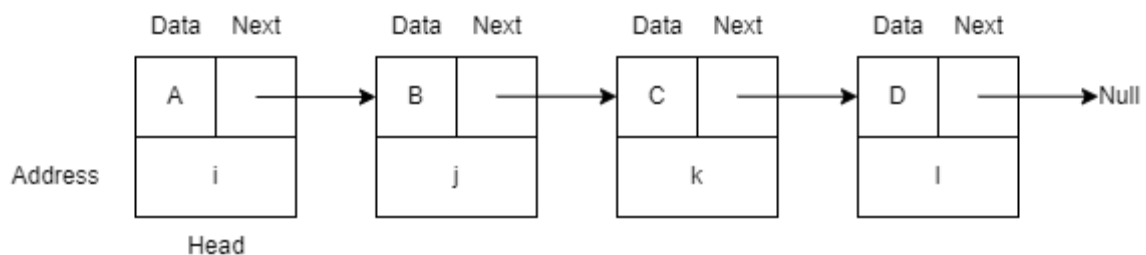
II. Materi

II.1. Linked List

Linked list merupakan salah satu struktur data yang mampu menyimpan banyak data. Struktur data ini secara sekilas mirip dengan *array* namun terdapat perbedaan terkait bagaimana data disimpan dalam memori. Sebuah *array* akan menyimpan data pada memori dengan alamat yang berurutan sedangkan *linked list* akan menyimpan data pada alamat memori yang acak. Ilustrasi untuk kedua struktur data tersebut dapat dilihat pada Gambar 17 dan Gambar 18.

	Elemen Pertama				Elemen Terakhir
Elemen	25	16	362	...	484
Address	x	x + 1	x + 2	...	x + n
Index	0	1	2		n

Gambar 17 Ilustrasi Array dengan Alamat Memori



Gambar 18 Ilustrasi Linked List dengan Alamat Memori

Perhatikan dari ilustrasi di atas terlihat bahwa struktur data *array* akan menyimpan masing-masing datanya secara urut pada alamat memori. Sebaliknya, struktur data *linked list* akan menyimpan data pada alamat memori yang acak namun tetap terkoneksi. Masing-masing blok data pada *linked list* sering disebut sebagai node. Sebuah node merupakan tipe data bentukan yang terdiri dari data dan *pointer* ke node berikutnya. Node pertama atau yang paling atas biasa disebut juga sebagai head.

Linked list memiliki beberapa tipe berdasarkan ikatan antar node. Dalam modul ini akan dijelaskan tiga jenis yang umum dijumpai yaitu *singly linked list*, *doubly linked list*, dan *circular linked list*. Penjelasan untuk masing-masing jenis *linked list* dapat dilihat pada poin berikut.

1. *Singly Linked List*

Singly linked list merupakan tipe *linked list* yang hubungan antar node hanya berlaku satu arah dari head hingga akhir. Gambar 18 merupakan tipe *singly linked list*. *Linked list* tipe ini paling umum digunakan.

2. *Doubly Linked List*

Doubly linked list merupakan tipe *linked list* yang mempunyai dua jalur hubungan node. Tipe *linked list* ini dapat diakses secara maju dari head hingga akhir ataupun sebaliknya.

3. *Circular Linked List*

Circular linked list merupakan tipe *linked list* yang node akhirnya memiliki jalur untuk kembali ke node head sehingga akses *linked list* tipe ini bisa diibaratkan seperti sebuah lingkaran.

Silakan pelajari lebih lanjut terkait *linked list* dengan mengakses referensi di bawah. Referensi [2] menjelaskan terkait operasi dasar yang sering dilakukan pada *linked list*. Pelajari juga tutorial yang diberikan pada bagian selanjutnya.

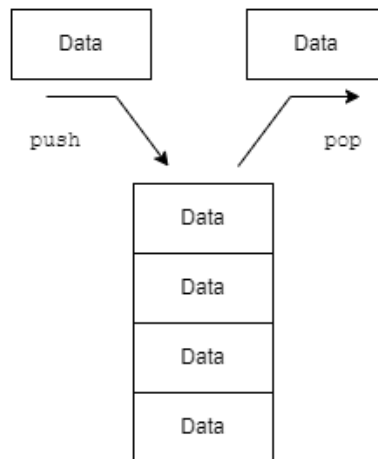
Referensi:

[1] https://www.tutorialspoint.com/data_structures_algorithms/linked_list_algorithms.htm 69

[2] https://www.tutorialspoint.com/dsa_using_c/dsa_using_c_linked_list.htm

II.II. *Stack*

Stack adalah salah satu tipe data abstrak yang sering digunakan pada berbagai Bahasa pemrograman. *Stack* akan menyimpan data dalam sebuah “tumpukan” yang hanya dapat diakses dari satu buah sisi saja. *Stack* dapat diibaratkan seperti tumpukan buku dalam kardus. Buku tersebut hanya dapat diambil dari tumpukan teratas dan hanya dapat dimasukkan buku baru pada bagian teratas. Tipe data yang seperti ini masuk kategori LIFO atau *last in first out*.



Gambar 19 Ilustrasi Stack

Stack dapat diimplementasikan menggunakan *array*, *structure*, maupun *linked list*. Dalam modul ini akan digunakan implementasi *stack* dengan *linked list*. Terdapat beberapa operasi dasar yang sering digunakan dalam mengolah *stack* yaitu push, pop, peek, dan isEmpty. Masing-masing penjelasan dari operasi dasar diberikan pada bagian berikut.

1. *Push*

Operasi ini digunakan untuk menambahkan data baru ke dalam *stack* yang sudah ada.

2. *Pop*

Operasi ini digunakan untuk menghapus atau mengeluarkan data paling atas dari sebuah *stack*.

3. *Peek*

Operasi ini digunakan untuk mendapatkan nilai dari data paling atas dalam sebuah *stack*.

4. *isEmpty*

Operasi ini digunakan untuk melakukan pengecekan apakah suatu *stack* kosong.

Silakan pelajari lebih lanjut terkait *stack* pada referensi yang diberikan. Operasi dasar *stack* yang diberikan dapat dipelajari lebih lanjut pada tutorial yang tersedia di bagian selanjutnya.

Referensi:

[1] https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm

II.III. Queue

Queue merupakan salah satu jenis tipe data abstrak seperti *stack* yang sering dijumpai juga pada berbagai Bahasa pemrograman. Berbeda dengan *stack* yang berbentuk “tumpukan”, *queue* akan menyimpan datanya dalam bentuk “antrean”. *Queue* memiliki dua buah sisi yang dapat diakses yaitu front dan rear. Masing-masing sisi tersebut digunakan untuk jalur keluar dan masuknya data. Sebuah *queue* dapat diibaratkan sebagai antrean di kasir. Antrean tersebut akan memiliki dua buah sisi yaitu di dekat kasir untuk jalur keluar dan di sisi pelanggan untuk menerima antrean. Metode seperti ini sering disebut sebagai FIFO atau *first in first out*.



Gambar 20 Ilustrasi Queue

Queue dapat diimplementasikan menggunakan *array*, *structure*, maupun *linked list*. Dalam modul ini akan digunakan implementasi *stack* dengan *linked list*. Terdapat beberapa operasi dasar yang sering digunakan dalam mengolah *queue* yaitu enqueue, dequeue, peek dan isEmpty. Masing-masing penjelasan dari operasi dasar diberikan pada bagian berikut.

1. Enqueue

Operasi ini digunakan untuk menambahkan data baru ke dalam *queue* yang sudah ada.

2. Dequeue

Operasi ini digunakan untuk menghapus atau mengeluarkan data dari sebuah *queue*.

3. Peek

Operasi ini digunakan untuk mendapatkan nilai dari data paling atas dalam sebuah *queue*.

5. isEmpty

Operasi ini digunakan untuk melakukan pengecekan apakah suatu *queue* kosong.

Silakan pelajari lebih lanjut terkait *queue* pada referensi yang diberikan. Operasi dasar *queue* yang diberikan dapat dipelajari lebih lanjut pada tutorial yang tersedia di bagian selanjutnya.

Referensi:

[1] https://www.tutorialspoint.com/data_structures_algorithms/dsa_queue.htm

III. Tutorial

Di bawah ini tersedia kode tutorial yang terkait dengan materi pada modul ini. Silakan Anda menyalin kode berikut dan menjalankannya pada komputer. Cermati proses eksekusi program sambil membaca dan memahami komentar serta maksud dari kode yang diberikan.

Source Code 7 Kode Tutorial 1 Modul 4

```
/** EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul : 4 - Linked List
 * Percobaan : Tutorial
 * Hari dan Tanggal : Selasa, 23 Maret 2026
 * Nama File : linked-list.c
 * Pembuat : Stefen Sutandi
 * Deskripsi : Main file untuk tutorial modul 04.
 */

/** Perintah Kompilasi
 *
 * Linux/MacOS - gcc -o linked-list linked-list.c lib.c
 * Windows - gcc -o linked-list.exe linked-list.c lib.c
 *
 * atau dengan Makefile:
 *
 * Linux/MacOS - make
 */

#include <stdio.h>
#include <stdlib.h>
#include "lib.h"

int main() {
    // Tambahkan tiga node pertama linked list
    Node* head = (Node*) malloc(sizeof(Node));
    Node* second_node = (Node*) malloc(sizeof(Node));
    Node* third_node = (Node*) malloc(sizeof(Node));

    // Inisialisasi data pada node
    head->data = 9;
    second_node->data = 6;
    third_node->data = 7;

    // Hubungkan node satu dengan lainnya
    head->next = second_node;
    second_node->next = third_node;
    third_node->next = NULL;

    // Print isi linked list secara manual
    printf("%d %d %d \n", head->data, head->next->data, head->next->next->data);

    // Print isi linked list dengan fungsi
    printLinkedList(head);

    // Tambahkan node berisi 420 di depan linked list
    push(&head, 420);
    printLinkedList(head);

    // Tambahkan node berisi 1337 setelah second_node
    insert(second_node, 1337);
    printLinkedList(head);

    // Hapus second_node
```

```

deleteNode(&head, second_node);
printLinkedList(head);

// Hapus node pertama yang berisi 9
deleteKey(&head, 9);
printLinkedList(head);

// Hapus node pertama yang berisi 420
deleteKey(&head, 420);
printLinkedList(head);

// Bebaskan memori (ekuivalen dengan free)
destroy(&head);

return 0;
}

```

Source Code 8 Kode Tutorial 2 Modul 4

```

/** EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul : 4 - Stacks and Queues
 * Percobaan : Tutorial
 * Hari dan Tanggal : Jumat, 2 April 2026
 * Nama File : tutorial-07-stack.c
 * Pembuat : Stefen Sutandi
 * Deskripsi : Source code tutorial untuk modul 04, materi stack.
 */

/** Perintah Kompilasi
 *
 * Linux/MacOS - gcc -o tutorial-07-stack tutorial-07-stack.c stack_lib.c
 * Windows - gcc -o tutorial-07-stack.exe tutorial-07-stack.c stack_lib.c
 *
 * atau dengan Makefile:
 *
 * Linux/MacOS - make stack
 */

#include <stdio.h>
#include <stdlib.h>
#include "stack_lib.h"

int main() {
    // Catatan:
    // pop tidak harus return 0 ketika stack kosong.
    // Pada contoh ini dibuat demikian agar lebih mudah dipahami.
    // Anda bisa mengganti 0 dengan INT_MIN misalnya.

    // Inisialisasi stack
    Stack* stack = (Stack*) malloc(sizeof(Stack));
    stack->head = NULL;

    // Push elemen ke dalam stack
    push(stack, 1);
    push(stack, 2);
    push(stack, 3);

    // Print isi stack saat ini
    printf("Current stack content:\n");
    printStack(stack);

    // Operasi pop dan peek secara bertahap
    printf("\nPop, got: %d\n", pop(stack));
    printf("Front of stack: %d\n", peek(stack));
    printf("Current stack content:\n");
    printStack(stack);
}

```

```

printf("\nPop, got: %d\n", pop(stack));
printf("Front of stack: %d\n", peek(stack));
printf("Current stack content:\n");
printStack(stack);

printf("\nPop, got: %d\n", pop(stack));
printf("Front of stack: %d\n", peek(stack));
printf("Current stack content:\n");
printStack(stack);

// Bebaskan memori stack
free(stack);

return 0;
}

```

Source Code 9 Kode Tutorial 3 Modul 4

```

/** EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
 * Modul : 4 - Stacks and Queues
 * Percobaan : Tutorial
 * Hari dan Tanggal : Jumat, 2 April 2026
 * Nama File : tutorial-07-queue.c
 * Pembuat : Stefen Sutandi
 * Deskripsi : Source code tutorial untuk modul 04, materi queue.
 */

/** Perintah Kompilasi
 *
 * Linux/MacOS - gcc -o tutorial-07-queue tutorial-07-queue.c queue_lib.c
 * Windows - gcc -o tutorial-07-queue.exe tutorial-07-queue.c queue_lib.c
 *
 * atau dengan Makefile:
 *
 * Linux/MacOS - make queue
 */

#include <stdio.h>
#include <stdlib.h>
#include "queue_lib.h"

int main() {
    // Catatan:
    // dequeue tidak harus return 0 ketika queue kosong.
    // Pada contoh ini dibuat demikian agar lebih mudah dipahami.
    // Anda bisa mengganti 0 dengan INT_MIN misalnya.

    // Inisialisasi queue
    Queue* queue = (Queue*) malloc(sizeof(Queue));
    queue->head = NULL;
    queue->tail = NULL;

    // Tambahkan elemen ke queue
    enqueue(queue, 1);
    enqueue(queue, 2);
    enqueue(queue, 3);

    // Print elemen depan dan belakang queue
    printf("Front of queue: %d\n", front(queue));
    printf("Rear of queue: %d\n", rear(queue));
    printf("Current queue content:\n");
    printQueue(queue);

    // Operasi dequeue secara bertahap
    printf("\nDequeue, got: %d\n", dequeue(queue));
}

```

```
printf("Front of queue: %d\n", front(queue));
printf("Rear of queue: %d\n", rear(queue));
printf("Queue content post-dequeue:\n");
printQueue(queue);

printf("\nDequeue, got: %d\n", dequeue(queue));
printf("Front of queue: %d\n", front(queue));
printf("Rear of queue: %d\n", rear(queue));
printf("Queue content post-dequeue:\n");
printQueue(queue);

printf("\nDequeue, got: %d\n", dequeue(queue));
printf("Front of queue: %d\n", front(queue));
printf("Rear of queue: %d\n", rear(queue));
printf("Queue content post-dequeue:\n");
printQueue(queue);

// Bebaskan memori queue
free(queue);

return 0;
}
```

MODUL V : FOUNDATION OF ALGORITHM

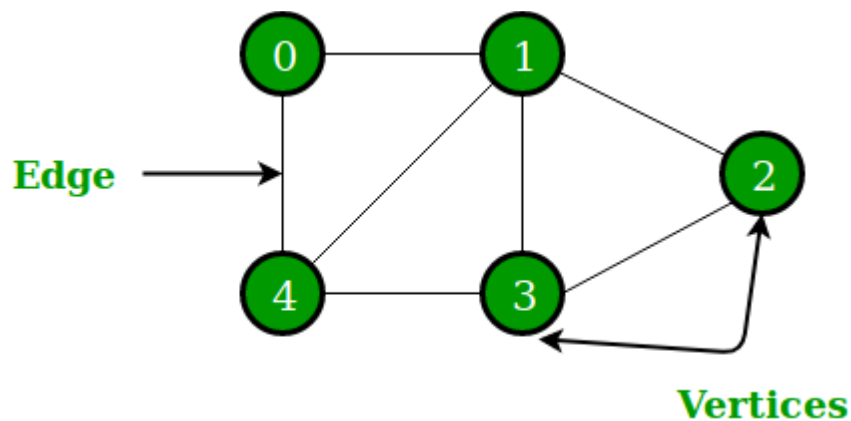
I. Tujuan

1. Mengasah keterampilan dalam **dekomposisi masalah** untuk merancang solusi yang lebih sistematis dan optimal.
2. Memberikan pemahaman mendalam mengenai **strategi pembuatan algoritma** yang efektif untuk menyelesaikan masalah kompleks.
3. Membekali praktikan dengan pemahaman menyeluruh mengenai **struktur data graph** dan **tree**, serta **algoritma umum** yang digunakan untuk mengelola dan memanipulasi struktur data tersebut.

II. Materi

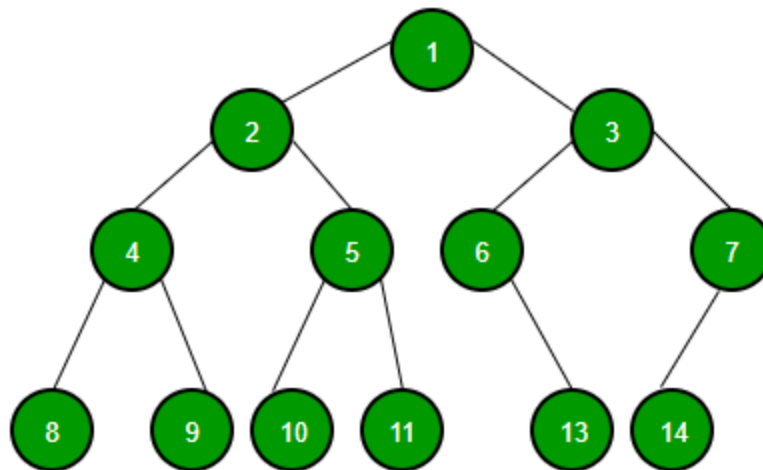
II.1. Graph and Tree

Graph dan *Tree* merupakan struktur data non-linear yang dapat menyimpan data. *Graph* dan *Tree* cukup sering digunakan dalam merepresentasikan data. *Graph* merupakan kumpulan dari *vertices* (atau *node*) yang terhubung melalui *edges*. Hubungan antar *vertices* melalui *edges* tidak terikat oleh sebuah aturan. Gambar 21 merupakan contoh dari sebuah graph.



Gambar 21 Ilustrasi Graph

Tree merupakan kumpulan *nodes* yang saling terhubung. Berbeda dengan *Graph*, *Tree* memiliki aturan hubungan antar *nodes*. *Tree* memiliki sebuah *node* yang disebut *root* dan hubungan antar *node* hanya berjarak 1 level. Gambar 22 berikut merupakan contoh tree.



Gambar 22 Ilustrasi Tree

Untuk mengoperasikan *graph* dan *tree* terdapat dua algoritma yang sering digunakan yaitu *depth first search* (DFS) dan *breadth first search* (BFS). DFS, sesuai namanya, akan melakukan pencarian dengan cara masuk ke dalam sebuah cabang hingga ke *node* akhir kemudian berpindah ke cabang lainnya. Algoritma ini akan menggunakan *stack* untuk membantu mengingat *node* atau *vertex* mana yang akan dikunjungi selanjutnya apabila ditemukan jalan buntu. Khusus untuk struktur data *tree*, algoritma DFS masih dapat dibagi lagi ke dalam tiga jenis metode berdasarkan urutan aksesnya. Metode tersebut adalah *pre-order*, *in-order*, dan *post-order*. Berikut ini diberikan contoh alur pencarian menggunakan algoritma DFS untuk ilustrasi *graph* dan *tree* di atas.

```

Graph
0-1-2-3-4
0-1-4-3-2
0-4-3-2-1
0-4-3-1-2
0-4-1-2-3
0-4-1-3-2
Tree
1-2-4-8
1-2-4-9
1-2-5-10
1-2-5-11
1-3-6-13
1-3-7-14
  
```

Algoritma BFS akan melakukan pencarian dengan cara menjelajahi seluruh *node* atau *vertex* pada “level” yang sama kemudian melanjutkan pencarian ke “level” berikutnya. Algoritma BFS akan memanfaatkan *queue* untuk menentukan *node* atau *vertex* selanjutnya yang akan dikunjungi apabila ditemukan jalan buntu. Berikut ini merupakan contoh alur pencarian menggunakan algoritma BFS pada ilustrasi *graph* dan *tree* sebelumnya.

```
Graph
0
1-4
2-3

Tree
1
2-3
4-5-6-7
8-9-10-11-13-14
```

Secara umum algoritma DFS dan BFS diimplementasikan menggunakan strategi *backtracking* secara rekursif. Silakan pelajari lebih lanjut terkait *graph* dan *tree* secara khusus DFS dan BFS pada referensi yang diberikan.

Referensi:

[1] https://www.tutorialspoint.com/dsa_using_c/dsa_using_c_graph.htm

[2] https://www.tutorialspoint.com/dsa_using_c/dsa_using_c_tree.htm

II.II. Strategi Algoritma

Algoritma merupakan prosedur sistematis yang digunakan untuk memecahkan suatu masalah. Dalam bahasan ini masalah yang diberikan akan diselesaikan menggunakan Bahasa C. Terdapat berbagai macam algoritma yang dapat digunakan untuk memecahkan berbagai masalah, namun pembahasan ini akan difokuskan pada strategi dasar yang mudah dipahami. Strategi dasar yang pertama adalah *backtracking* dan yang kedua adalah *divide and conquer*. Masing-masing strategi akan dibahas lebih lanjut pada poin berikut.

1. Backtracking

Backtracking adalah salah satu strategi untuk memecahkan masalah dengan cara mencoba berbagai kombinasi yang menjanjikan memberikan solusi dengan cara rekursif. Cara ini

merupakan cara yang paling mudah untuk dipahami dan digunakan, namun cara ini memiliki kekurangan yaitu kurang efektif karena harus melakukan pengecekan berulang.

2. *Divide and Conquer*

Divide and conquer merupakan salah satu strategi yang mudah dipahami dan cukup banyak digunakan. Strategi ini akan membagi sebuah masalah yang besar menjadi beberapa masalah kecil. Kemudian masing-masing masalah kecil tersebut diselesaikan dan solusi yang dihasilkan disusun ulang hingga menjadi solusi utuh untuk permasalahan yang diberikan.

Silakan Anda membaca lebih lanjut terkait strategi algoritma dan pelajari juga beberapa contoh yang diberikan pada referensi.

Referensi:

[1] <https://www.geeksforgeeks.org/backtracking-algorithms/>

[2] https://www.tutorialspoint.com/data_structures_algorithms/divide_and_conquer.htm

III. Tutorial

Di bawah ini tersedia kode tutorial yang terkait dengan materi pada modul ini. Kode yang diberikan merupakan contoh salah satu masalah dan proses penyelesaiannya. Silakan Anda menyalin kode berikut dan menjalankannya pada komputer. Cermati alur berpikir yang digunakan pada program sambil membaca dan memahami komentar serta maksud dari kode yang diberikan.

Source Code 10 Kode Tutorial 1 Modul 5

```
* EL2008 Praktikum Pemecahan Masalah dengan Pemrograman 2025/2026
* Modul: 5 - Algorithms
* Percobaan: Tutorial
* Hari dan Tanggal: Rabu, 7 April 2026
* Nama File: knapsack.c
* Pembuat: Stefen Sutandi
* Deskripsi: Penyelesaian 0/1 knapsack problem dengan backtracking.
*/

/**
 * Perintah Kompilasi
 * Linux/MacOS - gcc -o knapsack knapsack.c
 * Windows - gcc -o knapsack.exe knapsack.c
 * atau dengan Makefile:
 * Linux/MacOS - make
 */

#include <stdio.h>
#include <stdlib.h>
```

```

// Struct daftar barang
typedef struct ItemList {
    int* value; // Nilai barang
    int* weight; // Berat barang
    int data_amt; // Jumlah barang
} ItemList;

/** is_promising. Cek apakah solusi menjanjikan untuk ditelusuri lebih jauh.
 * Hitung batas atas keuntungan yang dapat diperoleh dengan menambah barang
 * yang bisa ditambah secara greedy dan menambah fraksi harga barang yang
 * tersisa bila ada.
 * Barang yang dicek untuk ditambah hanya barang yang memiliki indeks
 * setelah indeks barang yang diperiksa saat ini.
 * @param idx indeks barang yang diperiksa saat ini.
 * @param cur_value harga total barang saat ini.
 * @param cur_weight berat total barang saat ini.
 * @param best_value keuntungan terbaik yang telah ditemukan.
 * @param item_list list harga barang dan beratnya.
 * @param weight_limit batas berat barang.
 * @return 1 bila menjanjikan, 0 bila tidak.
 */
int is_promising(int idx, int cur_value, int cur_weight, int* best_value, ItemList*
item_list, int weight_limit) {
    // Bila berat melebihi batas, solusi tidak menjanjikan
    if (cur_weight >= weight_limit) return 0;
    // Ambil barang dengan indeks setelah idx secara greedy, catat keuntungan dan
    beratnya
    int it = idx + 1;
    float bound = cur_value;
    int weight_so_far = cur_weight;

    while (it < item_list->data_amt && weight_so_far + item_list->weight[it] <=
weight_limit) {
        weight_so_far += item_list->weight[it];
        bound += item_list->value[it];
        ++it;
    }

    // Bila masih ada sisa barang, tambahkan fraksi harga barang tersebut untuk
    // memenuhi ransel (seakan barang tersisa bisa dibagi)
    if (it < item_list->data_amt) {
        bound += (weight_limit - weight_so_far) * item_list->value[it] / (float)
item_list->weight[it];
    }
    // Bila batas atas keuntungan melebihi keuntungan terbesar saat ini, solusi
    menjanjikan
    return bound > *best_value;
}

/** knapsack_recursive. Selesaikan masalah knapsack secara rekursif
 * dengan algoritma backtracking.
 *
 * @param idx indeks barang yang diperiksa saat ini.
 * @param cur_value harga barang total pada kombinasi ini.
 * @param cur_weight berat barang total pada kombinasi ini.
 * @param item_combination kombinasi barang.
 * @param best_value keuntungan terbaik yang telah ditemukan.
 * @param best_combination kombinasi yang menghasilkan keuntungan terbaik.
 * @param item_list list harga barang dan beratnya.
 * @param weight_limit limit harga barang.
 * @result best_value dan best_combination menyimpan keuntungan terbaik dan
 * kombinasi yang menghasilkannya.
 */
void knapsack_recursive(int idx, int cur_value, int cur_weight, int*
item_combination, int* best_value, int* best_combination, ItemList* item_list, int
weight_limit) {
    // Cek apakah solusi saat ini merupakan terbaik

```

```

    if (cur_weight <= weight_limit && cur_value > *best_value) {
        // Set solusi saat ini sebagai solusi terbaik
        *best_value = cur_value;
        for (int i = 0; i < item_list->data_amt; ++i) {
            best_combination[i] = item_combination[i];
        }
    }
    // Bila solusi menjanjikan, lakukan rekursi penambahan barang
    if (is_promising(idx, cur_value, cur_weight, best_value, item_list,
weight_limit)) {
        // Eksplor solusi dengan menyertakan barang pada idx
        item_combination[idx + 1] = 1;
        knapsack_recursive(idx + 1, cur_value + item_list->value[idx + 1], cur_weight
+ item_list->weight[idx + 1], item_combination, best_value, best_combination,
item_list, weight_limit);

        // Backtrack, lanjutkan eksplor solusi tanpa menyertakan barang pada idx
        item_combination[idx + 1] = 0;
        knapsack_recursive(idx + 1, cur_value, cur_weight, item_combination,
best_value, best_combination, item_list, weight_limit);
    }
}
/** print_knapsack. Fungsi driver untuk menyelesaikan 0/1 knapsack problem.
 * @param item_list list harga dan berat barang.
 * @param weight_limit batas berat barang.
 * @result keuntungan terbanyak dan kombinasinya tercetak di layar.
 */
void print_knapsack(ItemList* item_list, int weight_limit) {
    int* item_combination = (int*) calloc(item_list->data_amt, sizeof(int));
    int* best_combination = (int*) calloc(item_list->data_amt, sizeof(int));
    int best_value = 0;

    knapsack_recursive(-1, 0, 0, item_combination, &best_value, best_combination,
item_list, weight_limit);

    printf("\nBest value: %d\n", best_value);
    printf("Best combination:\n");
    for (int i = 0; i < item_list->data_amt; ++i) {
        if (best_combination[i]) {
            printf("Item #%d, Value: %d, Weight: %d\n", i + 1, item_list->value[i],
item_list->weight[i]);
        }
    }
}

int main() {
    // Input barang dan batas berat secara dinamis
    ItemList item_list;
    int weight_limit;
    printf("Input data amount: ");
    scanf("%d", &(item_list.data_amt));
    item_list.value = (int*) malloc(item_list.data_amt * sizeof(int));
    item_list.weight = (int*) malloc(item_list.data_amt * sizeof(int));

    for (int i = 0; i < item_list.data_amt; ++i) {
        printf("\nInput item #%d value: ", i + 1);
        scanf("%d", &(item_list.value[i]));
        printf("Input item #%d weight: ", i + 1);
        scanf("%d", &(item_list.weight[i]));
    }
    printf("\nInput weight limit: ");
    scanf("%d", &weight_limit);

    // Selesaikan 0/1 knapsack problem
    print_knapsack(&item_list, weight_limit);
    return 0;
}

```